



图像分割总结

珠玉在前<https://github.com/luwill/Deep-Learning-Image-Segmentation/blob/master/README.md>

CV三大顶会：ICCV、ECCV、CVPR

▼ 目录

引言

1. 语义分割概述

2. 关键技术组件

2.1 编码器与分类网络

2.2 解码器与上采样

2.2.1 双线性插值

2.2.2 转置卷积

2.2.3 反池化

2.3 Skip Connection

2.4 Dilate Conv与多尺度

2.5 后处理技术

2.6 深监督

2.7 注意力机制

2.8 通用技术

2.8.1 损失函数

2.8.2 精度描述

2.8.3 归一化

3. 数据Pipeline

3.1 Torch数据读取模板

3.2 transform与数据增强

4. 模型与算法

4.1 LeNet&AlexNet

4.2 VGG

4.3 Segnet

4.4 Resnet

4.5 FCN

4.6 UNet

4.7 Deeplab系列

4.8 PSPNet

4.9 UNet++

4.10 Unet3+

4.11 U2net

4.12 Swin-Transformer

4.13 Swin-Unet

4.14 UTNet

4.15 nnUnet

4.16 TransUnet

- [4.17 TransDeepLab](#)
- [4.18 ConvNet（待施工）](#)
- [4.19 MedNeXt（待施工）](#)
- 5. 语义分割训练Tips
 - [5.1 PyTorch代码搭建方式](#)
 - 5.2 可视化方法
 - [5.2.1 Visdom](#)
 - [5.2.2 TensorBoard](#)

引言

图像分类、目标检测和图像分割是基于深度学习的计算机视觉三大核心任务。三大任务之间明显存在着一种递进的层级关系，图像分类聚焦于整张图像，目标检测定位于图像具体区域，而图像分割则是细化到每一个像素。基于深度学习的图像分割具体包括语义分割、实例分割和全景分割。语义分割的目的是要给每个像素赋予一个语义标签。语义分割在自动驾驶、场景解析、卫星遥感图像和医学影像等领域都有着广泛的应用前景。本文作为基于PyTorch的语义分割技术手册，对语义分割的基本技术框架、主要网络模型和技术方法提供一个实战性指导和参考。

1. 语义分割概述

图像分割主要包括语义分割（Semantic Segmentation）和实例分割（Instance Segmentation）。那语义分割和实例分割具体都是什么含义？二者又有什么区别和联系？语义分割是对图像中的每个像素都划分出对应的类别，即实现像素级别的分类；而类的具体对象，即为实例，那么实例分割不但要进行像素级别的分类，还需在具体的类别基础上区别开不同的个体。例如，图像有多个人甲、乙、丙，那边他们的语义分割结果都是人，而实例分割结果却是不同的对象。另外，为了同时实现实例分割与不可数类别的语义分割，相关研究又提出了全景分割（Panoptic Segmentation）的概念。全景分割是语义分割和实例分割的结合，即要对所有目标都检测出来，又要区分出同个类别中的不同实例。语义分割、实例分割和全景分割具体如图1（b）、（c）和（d）图所示。

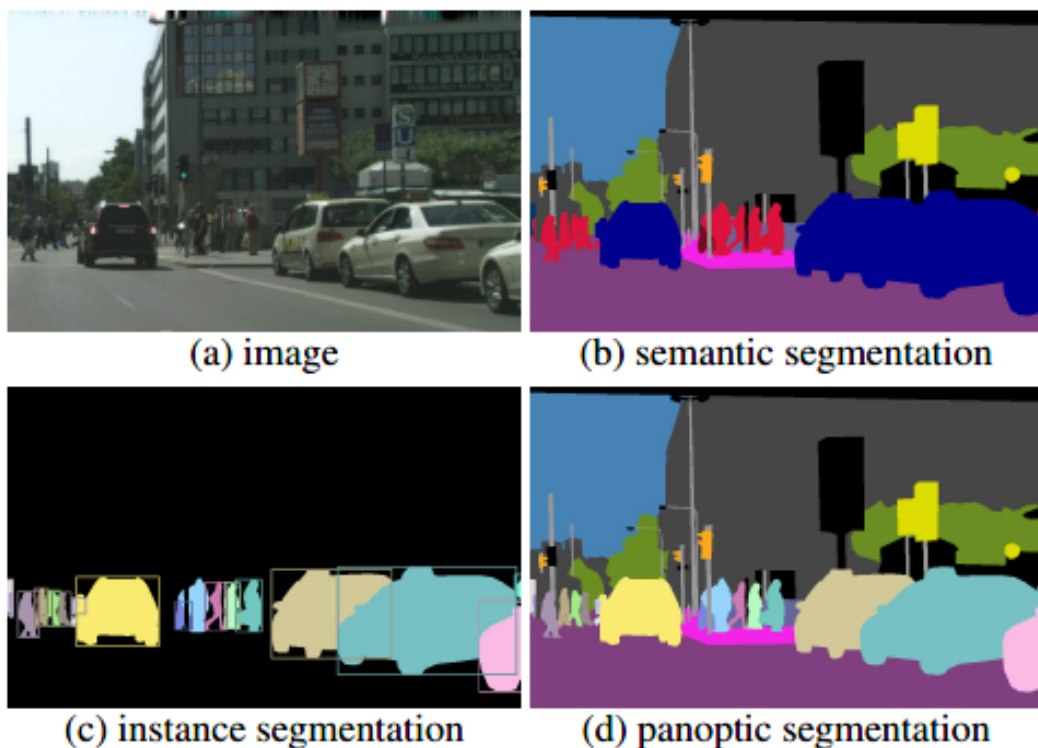


Fig1. Image Segmentation

在开始图像分割的学习和尝试之前，我们必须明确语义分割的任务描述，即搞清楚语义分割的输入输出都是什么。输入是一张原始的RGB图像或者单通道灰度图像，但是输出不再是简单的分类类别或者目标定位，而是带有各个像素类别标签的与输入同分辨率的分割图像。



在医学的图像分割中常用的图像有CT和MRI，一般来说输入都是灰度图，输出图大小一样，但是一般都是二分类（多器官之类的除外）即病灶和背景。

简单来说，我们的输入输出都是图像，而且是同样大小的图像。如图2所示。

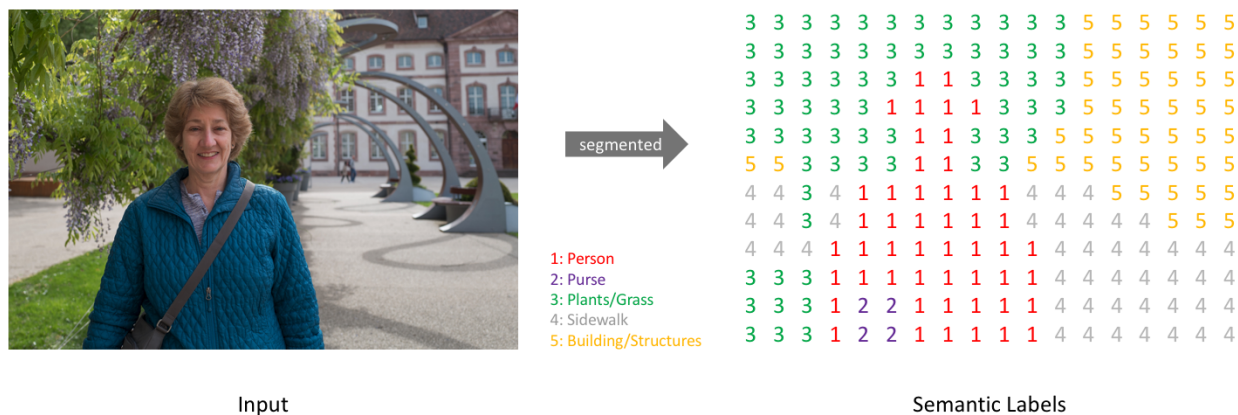


Fig2. Pixel Representation

类似于处理分类标签数据，对预测分类目标采用像素上的one-hot编码，即为每个分类类别创建一个输出的通道。如图3所示。

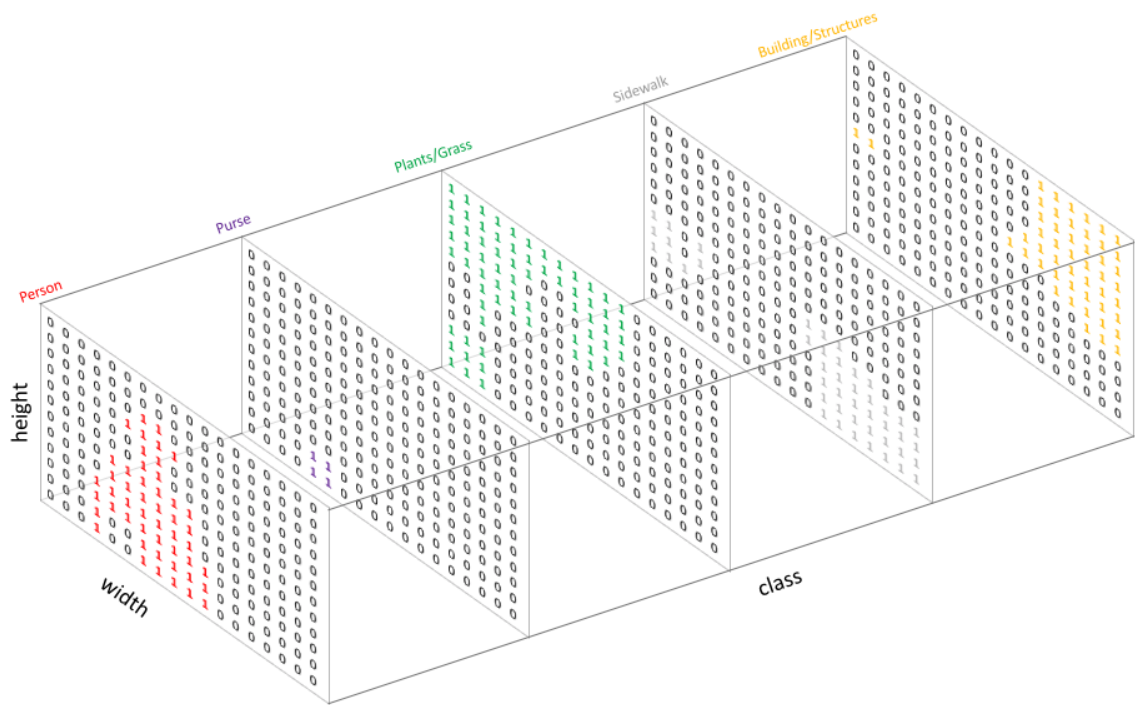


Fig3. Pixel One-hot

图4是将分割图添加到原始图像上的叠加效果。这里需要明确一下mask的概念，在图像处理中我们将其译为掩码，如Mask R-CNN中的Mask。Mask可以理解为我们将预测结果叠加到单个通道时得到的该分类所在区域。



Fig4. Pixel labeling

所以，语义分割的任务就是输入图像经过深度学习算法处理得到带有语义标签的同样尺寸的输出图像。

2. 关键技术组件

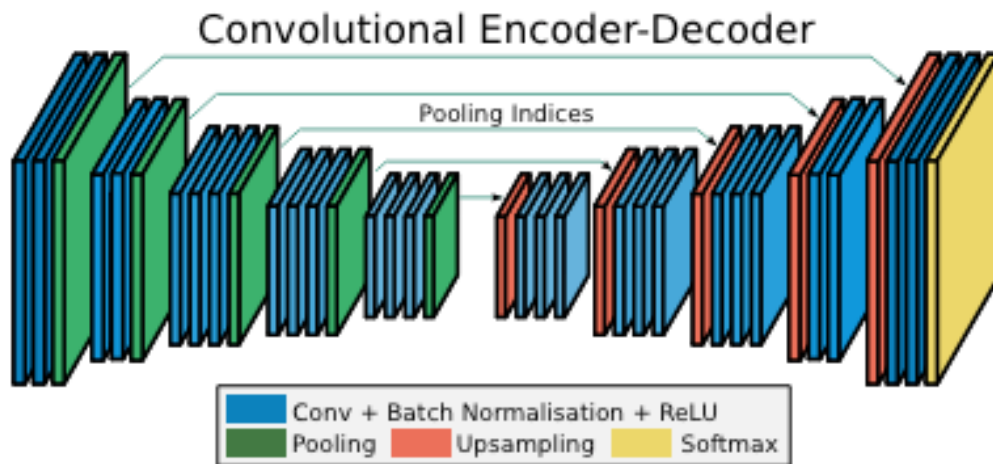
在语义分割发展早期，为了能够让深度学习进行像素级的分类任务，在分类任务的基础上对CNN做了一些修改，将分类网络中浓缩语义表征的全连接层去掉，提出用全卷积网络FCN（Fully Convolutional Networks）来处理语义分割问题。然后U-Net的提出，奠定了编解码结构的U形网络深度学习语义分割中的总山地位。这里我们对语义分割的关键技术组件进行分开描述，编码器、解码器和Skip Connection属于分割网络的核心结构组件，空洞卷积（Dilate Conv）是独立于U形结构的第二大核心设计。条件随机场（CRF）和马尔科夫随机场（MRF）则是用于优化神经网络分割后的细节处理。深监督作为一种常用的结构设计Trick，在分割网络中也有广泛应用。除此之外，则是针对于语义分割的通用技术点。

2.1 编码器与分类网络

编码器对于分割网络来说就是进行特征提取和语义信息浓缩的过程，这对熟悉各种分类网络的我们来说并不陌生。编码器通过卷积和池化的组合不断对图像进行下采样，得到的特征图空间尺寸也会越来越小，但会更加具备语义分辨性。这也是大多数分类网络的通用模式，不断卷积池化使得特征图越来越小，然后配上几层全连接网络即可进行分类判别。常用的分类网络包括AlexNet、VGG、ResNet、Inception、DenseNet和MobileNet等等。

既然之前有那么多优秀的SOTA网络用来做特征提取，所以很多时候分割网络的编码器并不需要我们write from scratch，时刻要有迁移学习的敏感度，直接用现成分类网络的卷积层部分作为编码器进行特征提取和信息浓缩，往往要比从头开始训练一个编码器要快很多。

比如我们以VGG16作为SegNet编码器的预训练模型，以PyTorch为例，来看编码器的写法。



```
class Encoder(nn.Module):
    def __init__(self, input_channels):
        super(Encoder, self).__init__()
        self.enco1 = nn.Sequential(
            nn.Conv2d(input_channels, 64, kernel_size=3, stride=1, padding=
1),
            nn.BatchNorm2d(64, momentum=bn_momentum),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1),
```

```

        nn.BatchNorm2d(64, momentum=bn_momentum),
        nn.ReLU()
    )
    self.enco2 = nn.Sequential(
        nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(128, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(128, momentum=bn_momentum),
        nn.ReLU()
    )
    self.enco3 = nn.Sequential(
        nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256, momentum=bn_momentum),
        nn.ReLU()
    )
    self.enco4 = nn.Sequential(
        nn.Conv2d(256, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU()
    )
    self.enco5 = nn.Sequential(
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU(),
        nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512, momentum=bn_momentum),
        nn.ReLU()
    )

```

```

def forward(self, x):
    id = [
        x = self.enco1(x)
        x, id1 = F.max_pool2d(x, kernel_size=2, stride=2, return_indices=True)
    e) # 保留最大值的位置
        id.append(id1)
        x = self.enco2(x)
        x, id2 = F.max_pool2d(x, kernel_size=2, stride=2, return_indices=True)
    e)
        id.append(id2)
        x = self.enco3(x)
        x, id3 = F.max_pool2d(x, kernel_size=2, stride=2, return_indices=True)
    e)
        id.append(id3)
        x = self.enco4(x)
        x, id4 = F.max_pool2d(x, kernel_size=2, stride=2, return_indices=True)
    e)
        id.append(id4)
        x = self.enco5(x)
        x, id5 = F.max_pool2d(x, kernel_size=2, stride=2, return_indices=True)
    e)
        id.append(id5)
    return x, id

```

在上述代码中，可以看到我们将vgg16的31个层分作5个编码模块，每个编码模块的基本结构如下所示：

```

Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
BatchNorm2d(64, momentum=bn_momentum)
ReLU(inplace=True)
Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
BatchNorm2d(64, momentum=bn_momentum)
ReLU(inplace=True)
MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

```

2.2 解码器与上采样

编码器不断将输入不断进行下采样达到信息浓缩，而解码器则负责上采样来恢复输入尺寸。解码器中除了一些卷积方法用为辅助之外，最关键的还是一些上采样方法，主要包括双线性插值、转置卷积和反池化。

2.2.1 双线性插值

插值法（Interpolation）是一种经典的数值分析方法，一些经典插值大家或多或少都有听到过，比如线性插值、三次样条插值和拉格朗日插值法等。在说双线性插值前我们先来了解一下什么是线性插值（Linear interpolation）。线性插值法是指使用连接两个已知量的直线来确定在这两个已知量之间的一个未知量的值的

方法。如下图所示：

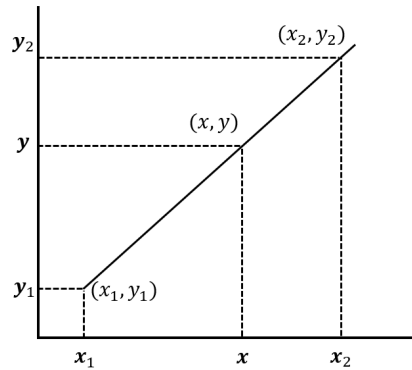


Fig5. Linear Interpolation

已知直线上两点坐标分别为 (x_1, y_1) 和 (x_2, y_2) ，现在想要通过线性插值法来得到某一点 x 在直线上的值。基本就是一个初中数学题，这里就不做过多展开，点 x 在直线上的值 y 可以表示为：

$$y = \frac{x_2 - x}{x_2 - x_1} y_1 + \frac{x - x_1}{x_2 - x_1} y_2$$

再来看双线性插值。线性插值用到两个点来确定插值，双线性插值则需要四个点。在图像上采样中，双线性插值利用四个点的像素值来确定要插值的一个像素值，其本质上还是分别在 x 和 y 方向上分别进行两次线性插值。如下图所示，我们来看具体做法。

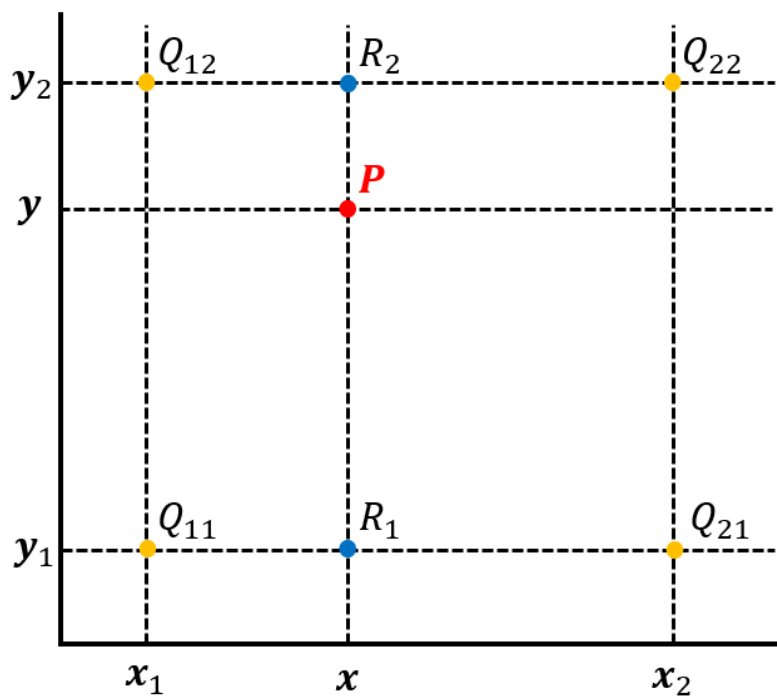


Fig6. Bilinear Interpolation

图中 Q_{11} - Q_{22} 四个黄色的点是已知数据点，红色点 P 是待插值点。假设 Q_{11} 为 (x_1, y_1) ， Q_{12} 为 (x_1, y_2) ， Q_{21} 为 (x_2, y_1) ， Q_{22} 为 (x_2, y_2) 。我们先在 x 轴方向上进行线性插值，先求得 R_1 和 R_2 的插值。根据线性插值公式，有：

$$f(R_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(R_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

得到 R_1 和 R_2 点坐标之后，便可继续在 y 轴方向进行线性插值。可得目标点 P 的插值为：

$$f(P) = \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2)$$

双线性插值在众多经典的语义分割网络中都有用到，比如说奠定语义分割编解码框架的FCN网络。假设将 3×6 的图像通过双线性插值变为 6×12 的图像，如下图所示。

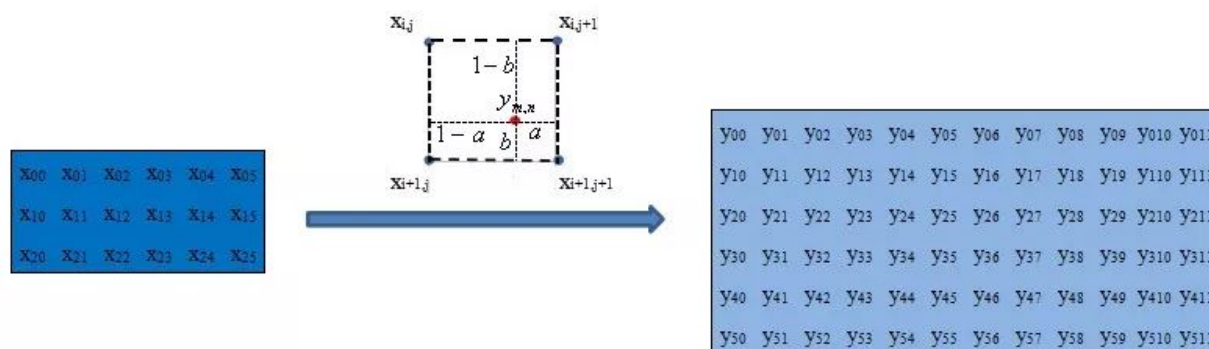


Fig7. Bilinear Interpolation Example

双线性插值的优点是速度非常快，计算量小，但缺点就是效果不是特别理想。

2.2.2 转置卷积

转置卷积（Transposed Convolution）也叫解卷积（Deconvolution），有些人也将其称为反卷积，但这个叫法并不太准确。大家都知道，在常规卷积时，我们每次得到的卷积特征图尺寸是越来越小的。但在图像分割等领域，我们是需要逐步恢复输入时的尺寸的。如果把常规卷积时的特征图不断变小叫做下采样，那么通过转置卷积来恢复分辨率的操作可以称作上采样。

本质上来说，转置卷积跟常规卷积并无区别。不同之处在于先按照一定的比例进行padding来扩大输入尺寸，然后把常规卷积中的卷积核进行转置，再按常规卷积方法进行卷积就是转置卷积。假设输入图像矩阵为 X ，卷积核矩阵为 C ，常规卷积的输出为 Y ，则有： $Y = CX$ ，两边同时乘以卷积核的转置 C^T ，这个公式便是转置卷积的输入输出计算。 $X = C^T Y$

假设输入大小为 4×4 ，滤波器大小为 3×3 ，常规卷积下输出为 2×2 ，为了演示转置卷积，我们将滤波器矩阵进行稀疏化处理为 4×16 ，将输入矩阵进行拉平为 16×1 ，相应输出结果也会拉平为 4×1 ，图示如下：

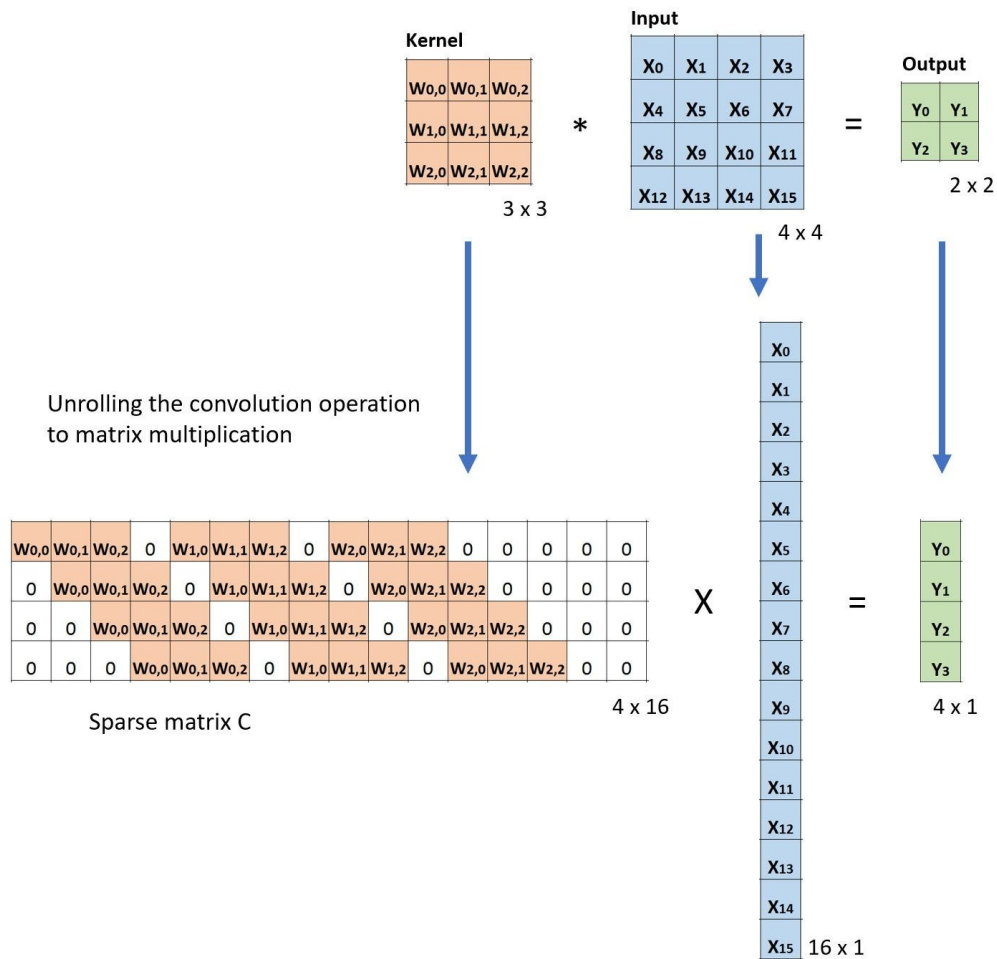


Fig8. Matrix of Convolution

然后按照转置卷积的做法我们把卷积核矩阵进行转置，按照 $X=CT^T$ 进行验证：

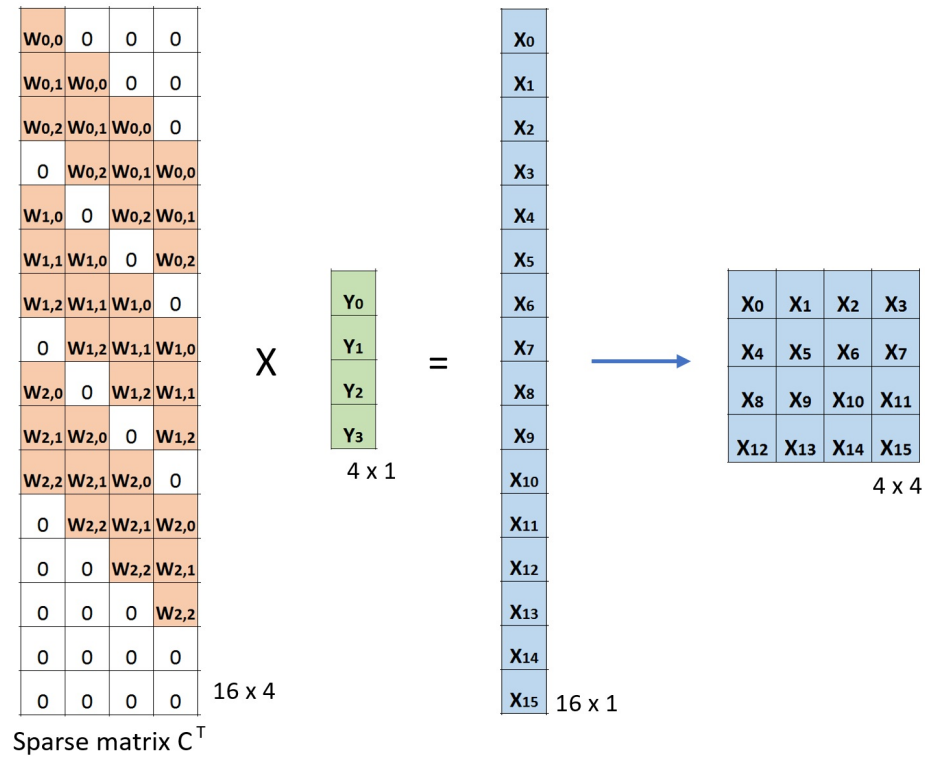
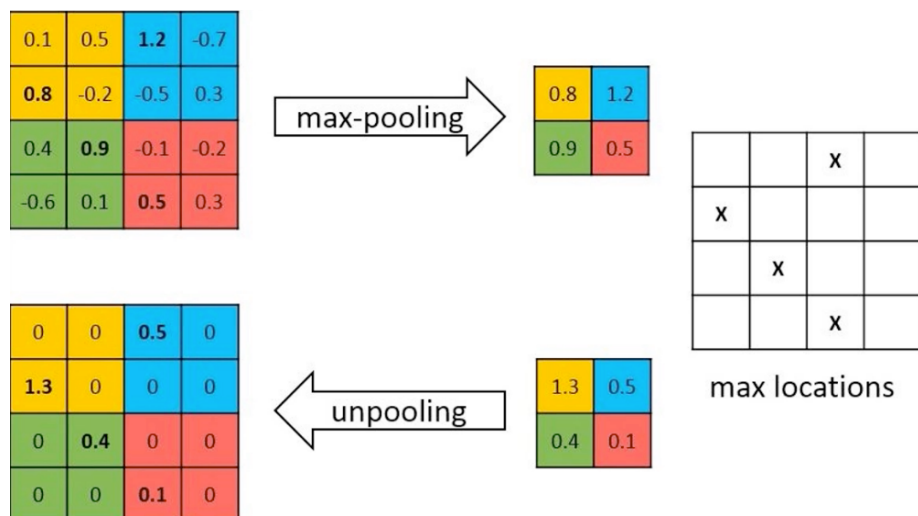


Fig9. Matrix of Transpose Convolution

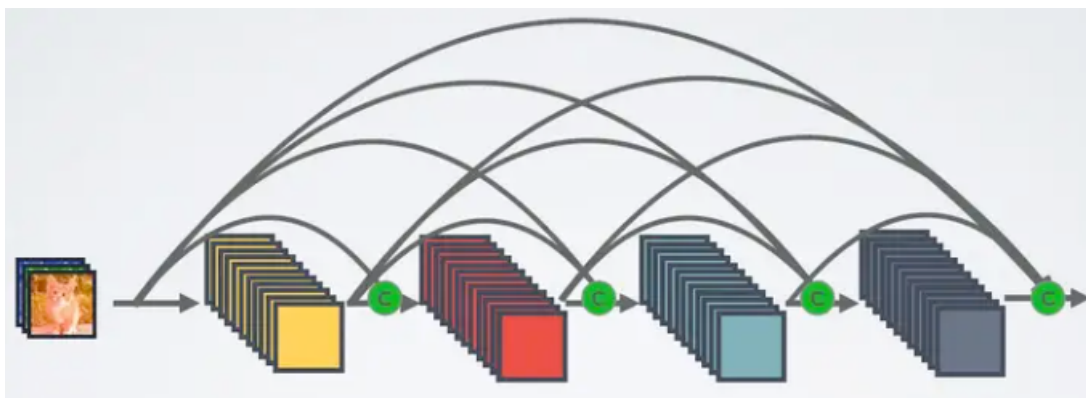
2.2.3 反池化

反池化（Unpooling）可以理解为池化的逆操作，相较于前两种上采样方法，反池化用的并不是特别多。其简要原理如下，在池化时记录下对应kernel中的坐标，在反池化时将一个元素根据kernel进行放大，根据之前的坐标将元素填写进去，其他位置补位为0即可。



2.3 Skip Connection

跳跃连接本身是在ResNet中率先提出，用于学习一个恒等式和残差结构，后面在DenseNet、FCN和U-Net等网络中广泛使用。最典型的就是U-Net的跳跃连接，在每个编码和解码层之间各添加一个跳跃连接，每一次下采样都会有一个跳跃连接与对应的上采样进行级联，这种不同尺度的特征融合对上采样恢复像素大有帮助。



2.4 Dilate Conv与多尺度

空洞卷积（Dilated/Atrous Convolution）也叫扩张卷积或者膨胀卷积，字面意思上来说就是在卷积核中插入空洞，起到扩大感受野的作用。空洞卷积的直接做法是在常规卷积核中填充0，用来扩大感受野，且进行计算时，空洞卷积中实际只有非零的元素起了作用。假设以一个变量 a 来衡量空洞卷积的扩张系数，则加入空洞之后的实际卷积核尺寸与原始卷积核尺寸之间的关系： $K = k + (k-1)(a-1)$

其中

k 为原始卷积核大小， a 为卷积扩张率（dilation rate）， K 为经过扩展后实际卷积核大小。除此之外，空洞卷积的卷积方式跟常规卷积一样。当 $a=1$ 时，空洞卷积就退化为常规卷积。 $a=1, 2, 4$ 时，空洞卷积示意图如下：

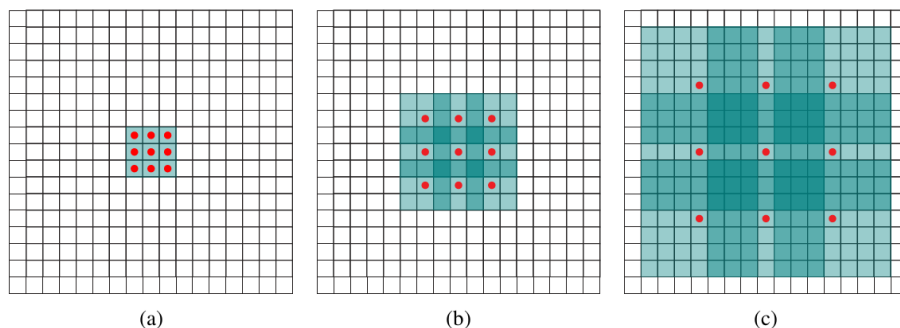


Fig10. Dialate Convolution

当 $a=1$ ，原始卷积核size为 3×3 ，就是常规卷积。 $a=2$ 时，加入空洞之后的卷积核size = $3 + (3-1) \times (2-1) = 5$ ，对应的感受野可计算为 $2(a+2) - 1 = 7$ 。 $a=3$ 时，卷积核size可以变化到 $3 + (3-1)(4-1) = 9$ ，感受野则增长到 $2(a+2) - 1 = 15$ 。对比不加空洞卷积的情况，在stride为1的情况下3层 3×3 卷积的叠加，第三层输出特征图对应的感受野也只有 $1 + (3-1) \times 3 = 7$ 。所以，空洞卷积的一个重要作用就是增大感受野。

在语义分割的发展历程中，增大感受野是一个非常重要的设计。早期FCN提出以全卷积方式来处理像素级别的分割任务时，包括后来奠定语义分割baseline地位的U-Net，网络结构中存在大量的池化层来进行下采样，大量使用池化层的结果就是损失掉了一些信息，在解码上采样重建分辨率的时候肯定会有影响。特别是对于多目

标、小物体的语义分割问题，以U-Net为代表的分割模型一直存在着精度瓶颈的问题。而基于增大感受野的动机背景下就提出了以空洞卷积为重大创新的deeplab系列分割网络。

对于语义分割而言，空洞卷积主要有三个作用：

第一是扩大感受野，具体前面已经说的比较多了，这里不做重复。但需要明确一点，池化也可以扩大感受野，但空间分辨率降低了，相比之下，空洞卷积可以在扩大感受野的同时不丢失分辨率，且保持像素的相对空间位置不变。简单而言就是空洞卷积可以同时控制感受野和分辨率。

第二就是获取多尺度上下文信息。当多个带有不同dilation rate的空洞卷积核叠加时，不同的感受野会带来多尺度信息，这对于分割任务是非常重要的。

第三就是可以降低计算量，不需要引入额外的参数，如上图空洞卷积示意图所示，实际卷积时只有带有红点的元素真正进行计算。

2.5 后处理技术

早期语义分割模型效果较为粗糙，在没有更好的特征提取模型的情况下，研究者们便在神经网络模型的粗糙结果进行后处理（Post-Processing），主要方法就是一些常用的概率图模型，比如说条件随机场（Conditional Random Field, CRF）和马尔可夫随机场（Markov Random Field, MRF）。

CRF是一种经典的概率图模型，简单而言就是给定一组输入序列的条件下，求另一组输出序列的条件概率分布模型，CRF在自然语言处理领域有着广泛应用。CRF在语义分割后处理中用法的基本思路如下：对于FCN或者其他分割网络的粗粒度分割结果而言，每个像素点 i 具有对应的类别标签 x_i 和观测值 y_i ，以每个像素为节点，以像素与像素之间的关系作为边即可构建一个CRF模型。在这个CRF模型中，我们通过观测变量 y_i 来预测像素 i 对应的标签值 x_i 。

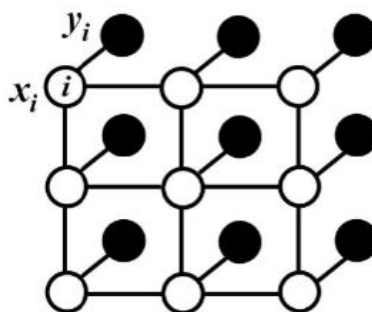


Fig11. CRF

以上做法也叫DenseCRF，具体细节可参考论文：[Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials](#)，除此之外还有CRFasRNN，采用平均场近似的方式将CRF方法融入到神经网络过程中，本质上是对DenseCRF的一种优化。

另一种后处理概率图模型是MRF，MRF与CRF较为类似，只是对CRF的二元势函数做了调整，其优点在于可以使用平均场来构造CNN网络，并且推理过程可以一次性搞定。MRF在Deep Parsing Network(DPN)中有详细描述，相关细节可参考论文[Semantic Image Segmentation via Deep Parsing Network](#)。

语义分割发展前期，在分割网络模型的结果上加上CRF和MRF等后处理技术形成了早期的语义分割技术框架：

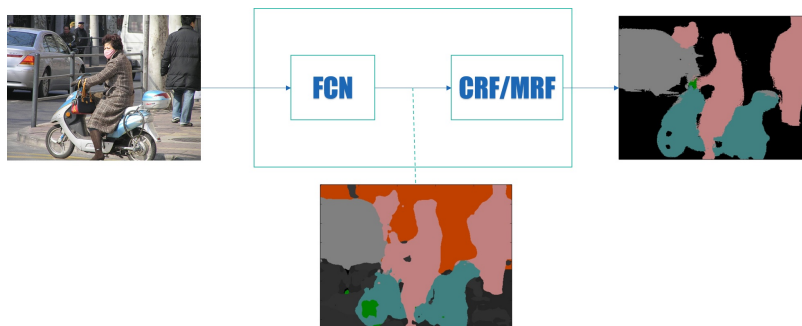


Fig12. Framework of Semantic Segmentation with CRF/MRF

但从Deeplabv3开始，主流的语义分割网络就不再热衷于后处理技术了。一个典型的观点认为神经网络分割效果不好才会用后处理技术，这说明在分割网络本身还有很大的提升空间。一是CRF本身不太容易训练，二来语义分割任务的端到端趋势。后来语义分割领域的SOTA网络也确实证明了这一点。尽管如此，CRF等后处理技术作为语义分割发展历程上的一个重要方法，我们有必要在此进行说明。从另一方面看，深度学习和概率图的结合虽然并不是那么顺利，但相信未来依旧会大有前景。

2.6 深监督

所谓深监督（Deep Supervision），就是在深度神经网络的某些中间隐藏层加了一个辅助的分类器作为一种网络分支来对主干网络进行监督的技巧，用来解决深度神经网络训练梯度消失和收敛速度过慢等问题。

带有深监督的一个8层深度卷积网络结构如下图所示。

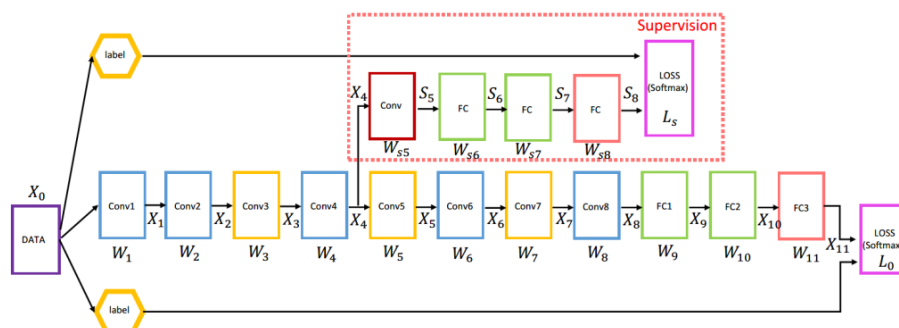


Fig13. Deep Supervision Example

可以看到，图中在第四个卷积块之后添加了一个监督分类器作为分支。**Conv4** 输出的特征图除了随着主网络进入 **Conv5** 之外，也作为输入进入了分支分类器。如图所示，该分支分类器包括一个卷积块、两个带有 **Dropout** 和 **ReLU** 的全连接块和一个纯全连接块。带有深监督的卷积模块例子如下。

```
class C1DeepSup(nn.Module):
    def __init__(self, num_class=150, fc_dim=2048, use_softmax=False):
        super(C1DeepSup, self).__init__()
        self.use_softmax = use_softmax
        self.cbr = conv3x3_bn_relu(fc_dim, fc_dim // 4, 1)
```

```

        self.cbr_deepsup = conv3x3_bn_relu(fc_dim // 2, fc_dim // 4, 1)
        # 最后一层卷积
        self.conv_last = nn.Conv2d(fc_dim // 4, num_classes, 1, 1, 0)
        self.conv_last_deepsup = nn.Conv2d(fc_dim // 4, num_class, 1, 1, 0)
        # 前向计算流程
        def forward(self, conv_out, segSize=None):
            conv5 = conv_out[-1]
            x = self.cbr(conv5)
            x = self.conv_last(x)
            # is True during inference
            if self.use_softmax:
                x = nn.functional.interpolate(
                    x, size=segSize, mode='bilinear', align_corners=False)
                x = nn.functional.softmax(x, dim=1)
            return x
        # 深监督模块
        conv4 = conv_out[-2]
        _ = self.cbr_deepsup(conv4)
        _ = self.conv_last_deepsup(_)
        # 主干卷积网络softmax输出
        x = nn.functional.log_softmax(x, dim=1)
        # 深监督分支网络softmax输出
        _ = nn.functional.log_softmax(_, dim=
1)

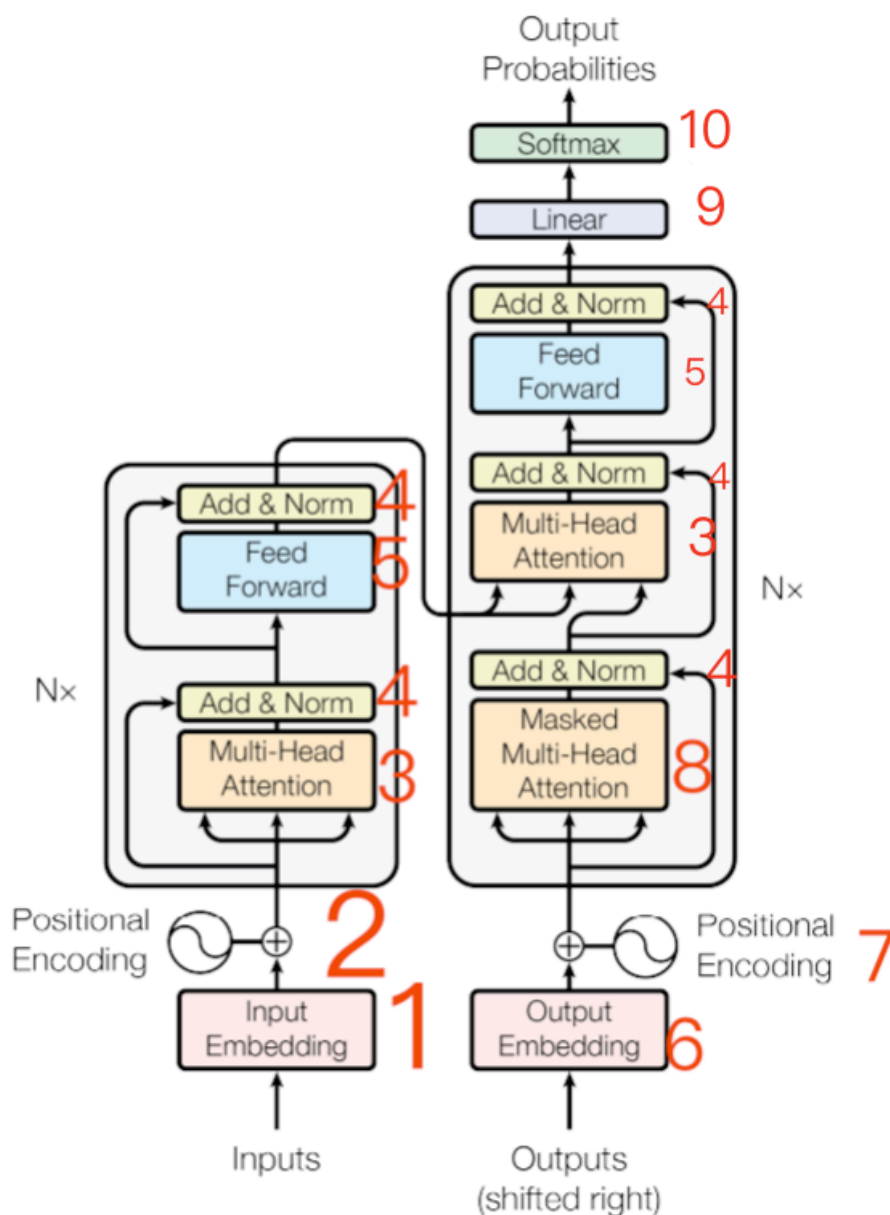
        return (x, _)

```

2.7 注意力机制

Transformer ([Attention Is All You Need](#) NIPS 2017) 是第一个完全依赖自注意力来计算输入和输出表示的传导模型，注意力机制是其中提出的最主要架构。

注意力机制的核心理念是关注重要信息，忽略次要信息。过程简单说就是在原来的weight参数基础上加一个权重分布的参数（也就是重要性），从而达到一个区别对待的效果，解决了RNN等模型在输入过长时导致对前面输入忽略的问题。



这是transformer模型的架构图，其中1是输入编码，2是位置编码，345称作一个编码器Encoder，3是多头注意力层，4是求和+归一化，5是前向传播，其中2到4，4到4还有一个残差使其适用于深度网络。右边6是之前的输入，7是位置编码，8是掩蔽多头注意力，7-9的部分称作一个解码器Decoder，9是线性变换，10是Softmax归一化后输出概率最大的单词。论文里的推荐设定是六个编码器六个解码器，即Input→Encoder。。。→Encoder→Decoder。。。→Decoder→Output的设定。

1、6:因为transformer的架构是应用于机器翻译方向，所以对输入的字符合要调用nn的embedding方法进行编码（Transformer模型是基于seq2seq的一个改进，但注意力机制是不分模型通用的）。

注意力机制通过动态计算输入序列中每个元素对当前输出的贡献，使得模型聚焦最关键的部分。而自注意力机制在处理序列时，直接在序列内部进行注意力权重的计算。

区别来说注意力机制通常用于模型的不同部分直接（比如编码器和解码器之间）的信息聚焦，而自注意力用于序列内部元素的相互作用。注意力机制需要依赖另一个序列（比如编码器的输出）来计算注意力权重，而自注

注意力机制仅使用序列内部的信息。自注意力机制可以更加直接捕捉序列内部的长距离依赖关系，而传统注意力机制侧重于捕捉两个序列直接的相关性。

CNN网络在提取底层特征和视觉结构方面有一定的优势。这些底层特征构成了在patch level上的关键点、线和一些基本的图像结构。这些底层特征具有明显的几何特性，往往关注诸如平移、旋转等变换下的一致性或者说是共变性。比如，一个CNN卷积滤波器检测得到的关键点、物体的边界等构成视觉要素的基本单元在平移等空间变换下应该是同时变换（共变性）的。CNN网络在处理这类共变性时是很自然的选择。

但当我们检测得到这些基本视觉要素后，高层的视觉语义信息往往更关注这些要素之间如何关联在一起进而构成一个物体，以及物体与物体之间的空间位置关系如何构成一个场景，这些是我们更加关心的。目前来看，transformer在处理这些要素之间的关系上更自然也更有效。

2.8 通用技术

通用技术主要是指深度学习流程中会用到的基本模块，比如说损失函数的选取以及采用哪种精度衡量指标。其他的像优化器的选取，学习率的控制等，这里限于篇幅进行省略。

2.8.1 损失函数

常用的分类损失均可用作语义分割的损失函数。最常用的就是交叉熵损失函数，如果只是前景分割，则可以使用二分类的交叉熵损失（Binary CrossEntropy Loss, BCE loss），对于目标物体较小的情况我们可以使用Dice损失，对于目标物体类别不均衡的情况可以使用加权的交叉熵损失（Weighted CrossEntropy Loss, WCE Loss），另外也可以尝试多种损失函数的组合。

2.8.2 精度描述

语义分割作为经典的图像分割问题，其本质上还是一种图像像素分类。既然是分类，我们就可以使用常见的分类评价指标来评估模型好坏。语义分割常见的评价指标包括像素准确率（Pixel Accuracy）、平均像素准确率（Mean Pixel Accuracy）、平均交并比（Mean IoU）、频权交并比（FWIoU）和Dice系数（Dice Coefficient）等。

像素准确率(PA)。

像素准确率跟分类中的准确率含义一样，即所有分类正确的像素数占全部像素的比例。PA的计算公式如下：

$$PA = \frac{\sum_{i=0}^k p_{ii}}{\sum_{i=0}^k \sum_{j=0}^k p_{ij}}$$

平均像素准确率(MPA)。

平均像素准确率其实更应该叫平均像素精确率，是指分别计算每个类别分类正确的像素数占所有预测为该类别像素数比例的平均值。所以，从定义上看，这是精确率(Precision)的定义，MPA的计算公式如下：

$$MPA = \frac{1}{k+1} \sum_{i=0}^k \frac{p_{ii}}{\sum_{j=0}^k p_{ij}}$$

平均交并比(MIoU)。

交并比（Intersection over Union）的定义很简单，将标签图像和预测图像看成是两个集合，计算两个集合的交集和并集的比值。而平均交并比则是将所有类的IoU取平均。

MIoU的计算公式如下：

$$MIoU = \frac{1}{k+1} \sum_{i=0}^k \frac{p_{ii}}{\sum_{j=0}^k p_{ij} + \sum_{j=0}^k p_{ji} - p_{ii}}$$

频权交并比(FWIoU)。

频权交并比顾名思义，就是以每一类别的频率为权重和其IoU加权计算出来的结果。FWIoU的设计思想很明确，语义分割很多时候会面临图像中各目标类别不平衡的情况，对各类别IoU直接求平均不是很合理，所以考虑各类别的权重就非常重要了。FWIoU的计算公式如下：

$$FWIoU = \frac{1}{\sum_{i=0}^k \sum_{j=0}^k p_{ij}} \sum_{i=0}^k \frac{\sum_{j=0}^k p_{ij} p_{ii}}{\sum_{j=0}^k p_{ij} + \sum_{j=0}^k p_{ji} - p_{ii}}$$

Dice系数。

Dice系数是一种度量两个集合相似性的函数，是语义分割中最常用的评价指标之一。Dice系数定义为两倍的交集除以像素和，跟IoU有点类似，其计算公式如下：

$$s = \frac{2|X \cap Y|}{|X| + |Y|}$$

dice本质上跟分类指标中的F1-Score类似。作为最常用的分割指标之一，这里给出PyTorch的实现方式。

```
import torch
def dice_coef(pred, target):
    """    Dice = (2*|X & Y|)/ (|X|+ |Y|)          = 2*sum(|A*B|)/(sum(A^2)+s
um(B^2))    """    smooth = 1.    m1 = pred.view(-1).float()
    m2 = target.view(-1).float()
    intersection = (m1 * m2).sum().float()
    dice = (2. * intersection + smooth) / (torch.sum(m1*m1) + torch.sum(m2*m
2) + smooth)
    return dice
```

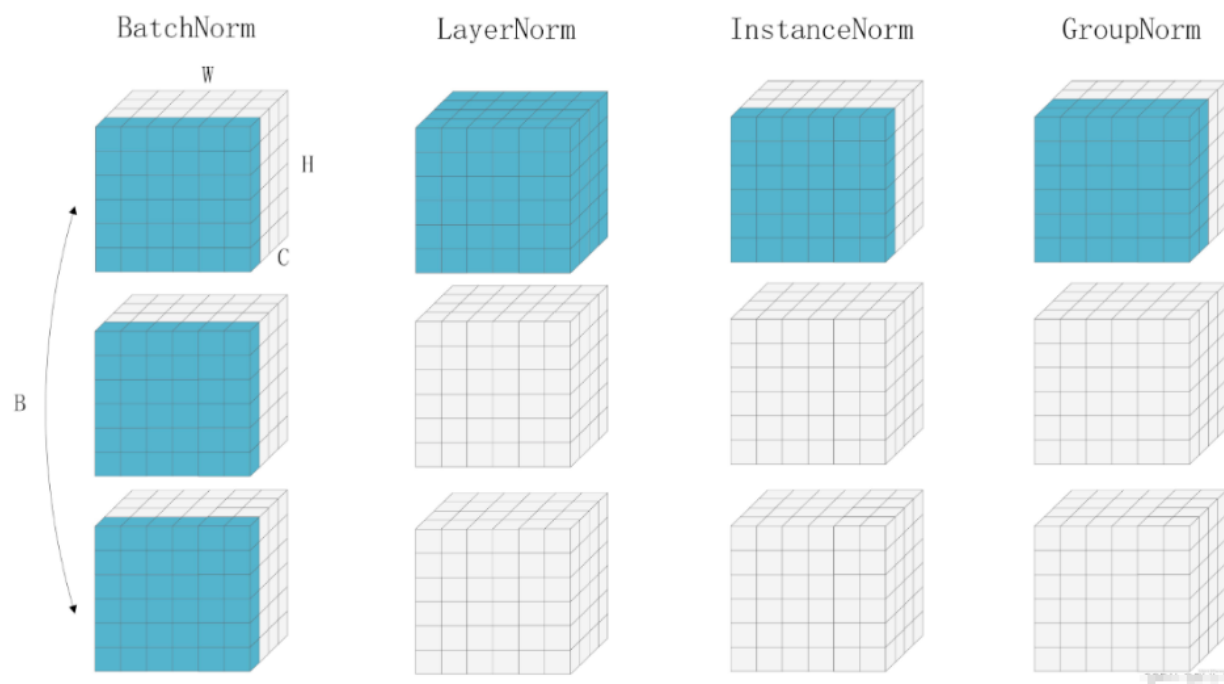
2.8.3 归一化



在训练神经网络时，往往需要标准化（Normalization）输入数据，使得网络的训练更加快速和有效，然而SGD等学习算法会在训练中不断改变网络的参数，隐含层的激活值的分布会因此发生变化，而这一种变化就称为内协变量偏移（Internal Covariate Shift, ICS）。

为了减轻ICS问题，Nomalization固定激活函数的输入变量的均值和方差（Nomalization后接激活函数），使得网络的训练更快。其计算方式大白话如下：将输入的数据，按照一定维度进行统计，得到数据的均值和方差，然后对数据结合均值和方差进行计算，将其变化为服从均值为0，方差为1的高斯分布，这样有助于加快网络训练速度。同时为了降低由于数据变化导致降低模型的表达能力，Nomalization提供了两个参数，用来对数据进行仿射变换，让这些数据靠近原始数据（最极限情况是这两个参数又将变换后的数据变为原来的数据）。

以下图片清晰的给出了不同归一化层的区别：



Batch Normalization (2015年提出)

Pytorch官网解

释:<https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html#torch.nn.BatchNorm2d>

原理:

针对输入到BN层的数据X，对所有 batch的单个通道做归一化，每个通道都分别做一次，公式如下：

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

其中：

$E[x]$ 是向量x的均值

$\text{Var}[x]$ 是向量x的方差

ϵ 为很小的常数，通常为0.00001，防止分母为0

γ 为仿射变换参数，模型可学习

β 为仿射变换参数，模型可学习

公式中gama之前的数据就是标准化后的数据，满足均值为0，方差为1的高斯分布，便于加快网络训练速度。

但是标准化有可能会降低模型的表达能力，因为网络中的某些隐藏层很有可能就是需要输入数据是非标准分布的，因此提供gama和beta进行学习，用于恢复网络的表达能力。

Pytorch代码:

```
torch.nn.BatchNorm2d(num_features, eps=1e-5, momentum=0.1, affine=True, track_r
```

num_features: 传入的通道数

eps: 常数 ϵ , 默认为0.00001

momentum: 动量参数, 用来控制均值和方差的更新

affine: 仿射变换的开关: 默认打开

如果 affine=False, 则 $\gamma=1$, $\beta=0$, 参数不能学习

如果 affine=True, 则 γ, β 可学习

track_running_stats: 如果为 True, 则统计跟踪 batch 的个数, 记录在 num_batches_tracked 中, 一般不用管。

示例:

```
import torch
import torch.nn as nn
input = torch.randn(20, 6, 10, 10)
m = nn.BatchNorm2d(6)
y = m(input)
print(y.shape)

out :
torch.Size([20, 6, 10, 10])
```

Layer Normalization (2016年提出)

Pytorch官网解

释:<https://pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html#torch.nn.LayerNorm>

原理:

针对输入到LN层的数据X, 对单个Batch中的所有通道数据做归一化, 然后每个batch都单独做一次, 公式如下:

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

参照Batch Normalization公式和最上面的图进行理解, 就是针对不同维度的数据进行标准化, 其他的没变。Transformer中使用的就是LayerNorm。

Pytorch代码:

```
torch.nn.LayerNorm(normalized_shape, eps=1e-5, elementwise_affine=True)
```

normalized_shape: 输入数据的维度（除了batch维度），例：数据维度【16, 64, 256, 256】传入的 normalized_shape维度为【64, 256, 256】。

eps：常数，默认值为0.00001

elementwise_affine：仿射变换的开关：默认打开

如果 elementwise_affine=False，则 $\gamma=1$ ， $\beta=0$ ，参数不能学习

如果 elementwise_affine=True，则 γ, β 可学习

示例:

```
1 import torch
2 import torch.nn as nn
3
4 ln = nn.LayerNorm([64, 256, 256])
5 ln2 = nn.LayerNorm(256)  # 给出最后一维数据即可
6 x = torch.randn(16, 64, 256, 256)
7 y = ln(x)
8 y2 = ln2(x)
9 print(y.shape)
10 print(y2.shape)
11
12
```

问题 输出 终端 端口 调试控制台

```
• (torch-env) PS E:\TNSCUI_Seg> python .\test.py
torch.Size([16, 64, 256, 256])
torch.Size([16, 64, 256, 256])
○ (torch-env) PS E:\TNSCUI_Seg>
```

Instance Normalization（2017年提出）

Pytorch官方解

释:<https://pytorch.org/docs/stable/generated/torch.nn.InstanceNorm2d.html#torch.nn.InstanceNorm2d>

原理:

针对输入到IN层的数据X，对单个Batch中的单个通道数据做归一化，然后每个batch的每个通道单独做一次，公式如下：

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

公式还是那个公式，理解还是那个理解，请看明白最上面的那种图，就清楚了。

Pytorch代码:

```
torch.nn.InstanceNorm2d(num_features, eps=1e-05, momentum=0.1, affine=False, tra
```

num_features: 是通道数

eps: 常数 ϵ ，默认为0.00001

momentum: 动量参数，用来控制均值和方差的更新

affine: 仿射变换的开关: 默认关闭

如果 affine=False, 则 $\gamma=1$, $\beta=0$, 参数不能学习

如果 affine=True, 则 γ, β 可学习

示例:

```
#Without Learnable Parameters
m = nn.InstanceNorm2d(100)
#With Learnable Parameters
m = nn.InstanceNorm2d(100, affine=True)
input = torch.randn(20, 100, 35, 45)
output = m(input)
print(output.shape)

out:
torch.Size([20, 100, 35, 45])
```

这个初始化只需要传入通道数即可，和BN的实例化方法一样，便于使用！

Group Normalization (2018年提出)

Pytorch官方解

释:<https://pytorch.org/docs/stable/generated/torch.nn.GroupNorm.html#torch.nn.GroupNorm>

原理:

针对输入到GN层的数据X，对单个Batch中的多个通道数据(一组数据)做归一化，然后每个batch的每组数据单独做一次，公式如下：

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

公式还是那个公式，理解还是那个理解，请看明白最上面的那种图，就清楚了。

Pytorch官网给出的四个公式也是一样的。

Pytorch代码:

```
torch.nn.GroupNorm(num_groups, num_channels, eps=1e-05, affine=True)
```

num_groups: 需要将Batch中的通道分为几组

num_channels: 传入的数据通道数

eps: 常数 ϵ , 默认为0.00001

affine: 仿射变换的开关: 默认打开

如果 affine=False, 则 $\gamma=1$, $\beta=0$, 参数不能学习

如果affine=True, 则 γ, β 可学习

示例:

```
import torch
import torch.nn as nn
input = torch.randn(20, 6, 10, 10)
m3 = nn.GroupNorm(3, 6) # 分成3组
m6 = nn.GroupNorm(6, 6) # 分成6组, 这个就和IN一样了
m1 = nn.GroupNorm(1, 6) # 分成1组, 这个就和LN一样了
y1 = m1(input)
y3 = m3(input)
y6 = m6(input)
print(y1.shape)
print(y3.shape)
print(y6.shape)

out:
torch.Size([20, 6, 10, 10])
torch.Size([20, 6, 10, 10])
torch.Size([20, 6, 10, 10])
```

3. 数据Pipeline

这里主要说一下PyTorch的自定义数据读取pipeline模板和相关tricks以及如何优化数据读取的pipeline等。我们从PyTorch的数据对象类 `Dataset` 开始。 `Dataset` 在PyTorch中的模块位于 `utils.data` 下。

```
from torch.utils.data import Dataset
```

3.1 Torch数据读取模板

PyTorch官方为我们提供了自定义数据读取的标准化代码代码模块, 作为一个读取框架, 我们这里称之为原始模板。其代码结构如下:

```

from torch.utils.data import Dataset
class CustomDataset(Dataset):
    def __init__(self, ...):
        # stuff      def __getitem__(self, index):
        # stuff      return (img, label)
    def __len__(self):
        # return examples size      return count

```

3.2 transform与数据增强

PyTorch数据增强功能可以放在 `transform` 模块下，添加 `transform` 后的数据读取结构如下所示：

```

from torch.utils.data import Dataset
from torchvision import transforms as T
class CustomDataset(Dataset):
    def __init__(self, ...):
        # stuff      # ...      # compose the transforms methods      s
        self.transform = T.Compose([T.CenterCrop(100),
                                    T.RandomResizedCrop(256),
                                    T.RandomRotation(45),
                                    T.ToTensor())])
    def __getitem__(self, index):
        # stuff      # ...      data = # Some data read from a file or im
        age      labe = # Some data read from a file or image      # execute the
        transform      data = self.transform(data)
        label = self.transform(label)
        return (img, label)
    def __len__(self):
        # return examples size      return count
if __name__ == '__main__':
    # Call the dataset      custom_dataset = CustomDataset(...)

```

需要说明的是，PyTorch `transform` 模块所做的数据增强并不是我们所理解的广义上的数据增强。`transform` 所做的增强，仅仅是在数据读取过程中随机地对某张图像做转化操作，实际数据量上并没有增多，可以将其视为是一种在线增强的策略。如果想要实现实际训练数据成倍数的增加，可以使用离线增强策略。

与图像分类仅需要对输入图像做增强不同的是，对于语义分割的数据增强而言，需要同时对输入图像和输入的mask同步进行数据增强工作。实际写代码时，要记得使用随机种子，在不失随机性的同时，保证输入图像和输出mask具备同样的转换。一个完整的语义分割在线数据增强代码实例如下：

```

import os
import random
import torch
from torch.utils.data import Dataset
from PIL import Image

```

```

class SegmentationDataset(Dataset):
    # read the input images    @staticmethod    def _load_input_image(path):
        with open(path, 'rb') as f:
            img = Image.open(f)
            return img.convert('RGB')
    # read the mask images    @staticmethod    def _load_target_image(path):
        with open(path, 'rb') as f:
            img = Image.open(f)
            return img.convert('L')
    def __init__(self, input_root, target_root, transform_input=None,
                 transform_target=None, seed_fn=None):
        self.input_root = input_root
        self.target_root = target_root
        self.transform_input = transform_input
        self.transform_target = transform_target
        self.seed_fn = seed_fn
        # sort the ids    self.input_ids = sorted(img for img in os.listdir(
ir(self.input_root)))
        self.target_ids = sorted(img for img in os.listdir(self.target_root))
        assert(len(self.input_ids) == len(self.target_ids))
    # set random number seed    def _set_seed(self, seed):
        random.seed(seed)
        torch.manual_seed(seed)
        if self.seed_fn:
            self.seed_fn(seed)
    def __getitem__(self, idx):
        input_img = self._load_input_image(
            os.path.join(self.input_root, self.input_ids[idx]))
        target_img = self._load_target_image(
            os.path.join(self.target_root, self.target_ids[idx]))
        if self.transform_input:
            # ensure that the input and output have the same randomness.
seed = random.randint(0, 2**32)
            self._set_seed(seed)
            input_img = self.transform_input(input_img)
            self._set_seed(seed)
            target_img = self.transform_target(target_img)
        return input_img, target_img, self.input_ids[idx]
    def __len__(self):
        return len(self.input_ids)

```

其中transform_input和transform_target均可由transform模块下的函数封装而成。一个皮肤病灶分割的在线数据增强实例效果如下图所示。

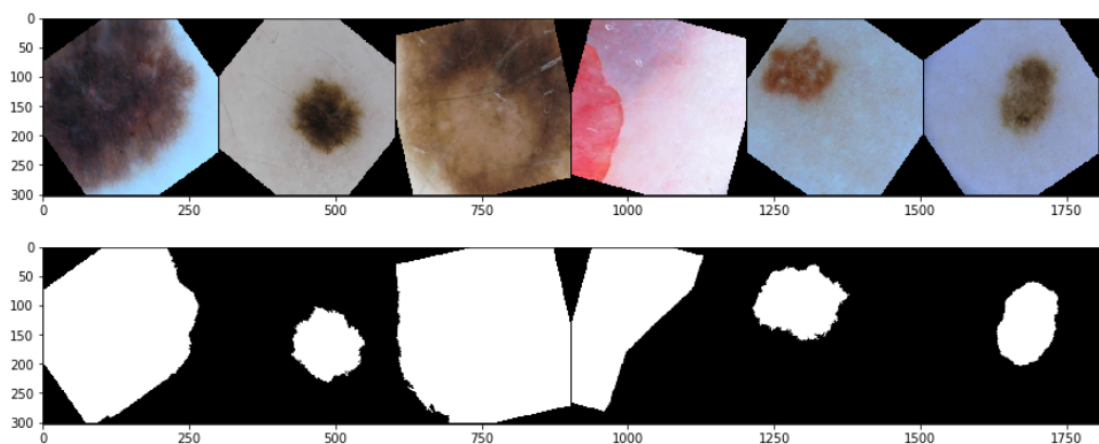


Fig14. Example of online augmentation

4. 模型与算法

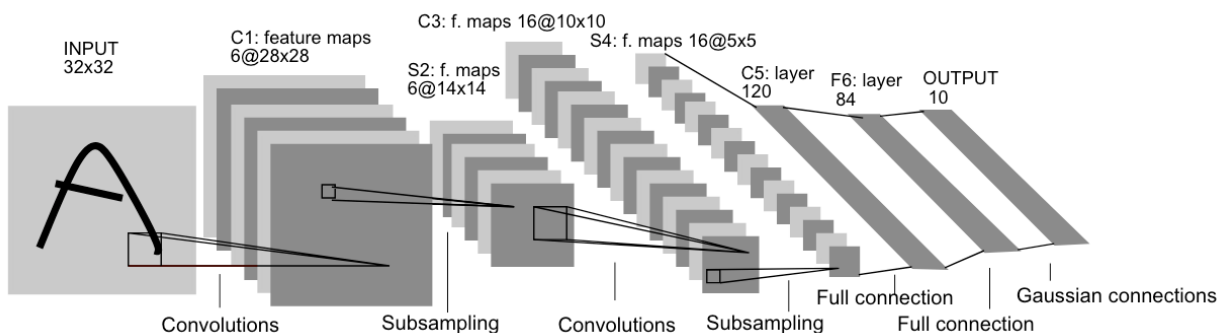
早期基于深度学习的图像分割以FCN为核心，旨在重点解决如何更好从卷积下采样中恢复丢掉的信息损失。后来逐渐形成了以U-Net为核心的这样一种编解码对称的U形结构。语义分割界迄今为止最重要的两个设计，一个是以U-Net为代表的U形结构，目前基于U-Net结构的创新就层出不穷，比如说应用于3D图像的V-Net，填充U-Net结构的U-Net++，嵌套Unet结构的U2net等。除此在外还有SegNet、RefineNet、HRNet和FastFCN。另一个则是以DeepLab系列为代表的Dilation设计，主要包括DeepLab系列和PSPNet。

随着模型的Baseline效果不断提升，语义分割任务的主要矛盾也逐从downsample损失恢复像素逐渐演变为如何更有效地利用context上下文信息。

本节将从CNN开始介绍各种模型一路发展过来的历程。

4.1 LeNet&AlexNet

CNN的开山之作是哪篇不重要，本文中CNN的发展从1998年的《[Gradient-Based Learning Applied to Document Recognition](#)》开始。



LeNet5包含Input、卷积层1、池化层1、卷积层2、池化层2、全连接层、输出层。LeNet-5共有7层（不包含输入），每层都包含可训练参数。输入图像大小为 32×32 ，比MNIST数据集的图片要大一些，这么做的原因是希望潜在的明显特征如笔画断点或角能够出现在最高层特征检测子感受野（receptive field）的中心。因此在训练整个网络之前，需要对 28×28 的图像加上padding（即周围填充0）。

C1层：该层是一个卷积层。使用6个大小为 5×5 的卷积核对输入层进行卷积运算，特征图尺寸为 $32 - 5 + 1 = 28$ ，因此产生6个大小为 28×28 的特征图。这么做够防止原图像输入的信息掉到卷积核边界之外。

S2层：该层是一个池化层（pooling，也称为下采样层）。通过计算其邻域四个像素的平均，然后将其乘以一个权重 w ，再加上一个偏置 b ，最后通过一个Sigmoid激活函数得到下采样的结果，池化的size定为 2×2 ，经池化后得到6个 14×14 的特征图，作为下一层神经元的输入。

C3层：该层仍为一个卷积层，我们选用大小为 5×5 的16种不同的卷积核。这里需要注意：C3中的每个特征图，都是S2中的所有6个或其中几个特征图进行加权组合得到的。输出为16个 10×10 的特征图。

S4层：该层仍为一个池化层，size为 2×2 ，平均采样。最后输出16个 5×5 的特征图，神经元个数也减少至 $16 \times 5 \times 5 = 400$ 。

C5层：该层我们继续用 5×5 的卷积核对S4层的输出进行卷积，卷积核数量增加至120。这样C5层的输出图片大小为 $5 - 5 + 1 = 1$ 。最终输出120个 1×1 的特征图。这里实际上是与S4全连接了，但仍将其标为卷积层，原因是如果LeNet-5的输入图片尺寸变大，其他保持不变，那该层特征图的维数也会大于 1×1 。

F6层：该层与C5层全连接，输出84张特征图。

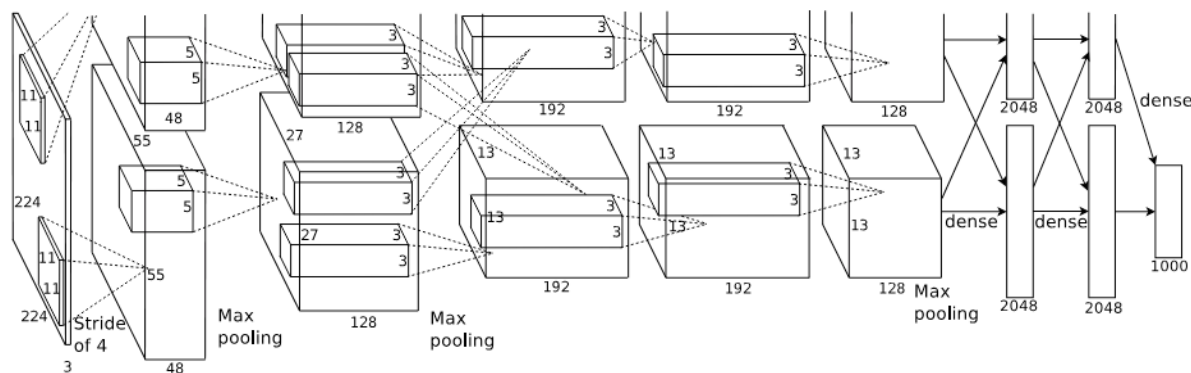
输出层：该层与F6层全连接，输出长度为10的张量，代表所抽取的特征属于哪个类别。（例如 $[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$ 的张量，1在 $\text{index} = 3$ 的位置，故该张量代表的图片属于第三类）

LeNet5当时的特性有如下几点：

- (1) 每个卷积层包含三个部分：卷积、池化和非线性激活函数
- (2) 使用卷积提取空间特征（起初被称为感受野，未提“卷积”二字）
- (3) 降采样（Subsample）的平均池化层（Average Pooling）
- (4) 双曲正切（Tanh）的激活函数
- (5) MLP作为最后的分类器
- (6) 使用卷积令层与层之间稀疏连接，减少了计算复杂性。

LeNet由CNN之父Lecun在1998年提出，用于解决手写数字识别任务。著名的Tensorflow入门学习数据集MNIST，就是那时候做出来的。限于当时计算力的不足，LeNet训练较慢，且还有一些其他算法（比如SVM）能达到类似或者更好的效果，LeNet并没有真正的火起来。

而AlexNet (ImageNet Classification with Deep Convolutional Neural Networks) 在2012年ImageNet竞赛中以超过第二名10.9个百分点的绝对优势一举夺冠，这一突破是具有重大历史意义，因为它使计算机视觉模型跨过了从学术demo到商业化产品的门槛，从而将深度学习和CNN的名声突破学术界，在产业界一鸣惊人。AlexNet的出现可谓是卷积神经网络的王者归来。AlexNet为8层深度的CNN网络，其中包括5个卷积层和3个全连接层（不包括LRN层和池化层）。



AlexNet的特点和贡献：

(1) 使用ReLU作为激活函数，由于ReLU是非饱和函数，也就是说它的导数在大于0时，一直是1，因此解决了Sigmoid激活函数在网络比较深时的梯度消失问题，提高SGD（随机梯度下降）的收敛速度。

(2) 使用Dropout方法避免模型过拟合，该方法通过让全连接层的神经元（该模型在前两个全连接层引入Dropout）以一定的概率失去活性（比如0.5），失活的神经元不再参与前向和反向传播，相当于约有一半的神经元不再起作用。在预测的时候，让所有神经元的输出乘Dropout值（比如0.5）。这一机制有效缓解了模型的过拟合。

(3) 重叠的最大池化，之前的CNN中普遍使用平均池化，而AlexNet全部使用最大池化，避免平均池化的模糊化效果。并且，池化的步长小于核尺寸，这样使得池化层的输出之间会有重叠和覆盖，提升了特征的丰富性。

(4) 提出LRN（局部响应归一化）层，对局部神经元的活动创建竞争机制，使得响应较大的值变得相对更大，并抑制其他反馈较小的神经元，增强了模型的泛化能力。

(5) 使用GPU加速，加快了模型的训练速度，同时也意味着可以训练更大规模的神经网络模型。AlexNet在双gpu上运行，每个gpu负责一半网络的运算。训练速度得到提升，大概是使用CPU的20~50倍；

(6) 数据增强，随机从256256的原始图像中截取224224大小的区域（以及水平翻转的镜像），相当于增强了 $(256-224) * (256-224) * 2 = 2048$ 倍的数据量。使用了数据增强后，减轻过拟合，提升泛化能力。避免因原始数据量的大小使得参数众多的CNN陷入过拟合中。

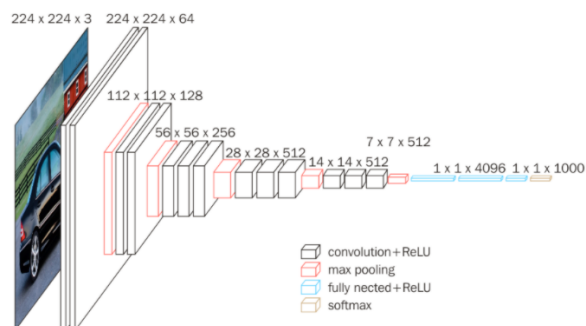
我们可以发现，前几个卷积层的计算量很大，但参数量很小，只占Alexnet总参数的很小一部分。这就是卷积层的优点——通过较小的参数量来提取有效的特征。

论文中指出，如果去掉任何一个卷积层，都会使网络的分类性能大幅下降。

4.2 VGG

VGG (Very Deep Convolutional Networks for Large-Scale Image Recognition 2014ILSVRC) 在Alexnet的基础上，使用了新架构——通过堆叠多个3*3卷积核替代一个大尺度卷积核，比传统CNN参数更少，减少了计算开销。

VGG模型是具有多层的标准深度卷积神经网络架构，其中“深”是指由16和19个卷积层组成的VGG-16或VGG-19的层数。VGG16在性能，效果上最均衡，是当时最好用的分类网络之一。



ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

在VGG16中，首先输入一个224*224的图像（固定大小），然后经过两个3*3的卷积层，深度为64，经过一次采样缩小一半变成112*112，再经过两个3*3的卷积层，经过一次采样缩小到56*56，再经过三个3*3的卷积层，采样降到28*28，再经过三个3*3的卷积层，采样缩到14*14，再经过三个3*3的卷积层，缩小到7*7，经过两个全连接层展开，然后映射到1000维度的矩阵（分类个数），最后经过softmax输出概率。

卷积层中卷积核不改变图像大小，采样将高和宽都降低一半。3*3的卷积核是捕捉左右，上下，中心概念的最小尺寸。

VGG特点：

- 1.VGG16相比AlexNet的一个改进是采用连续的3x3的卷积核代替AlexNet中的较大卷积核（11x11，7×7，5×5）
- 2.加深结构都使用ReLU激活函数：提升非线性变化的能力
- 3.VGG16 全部采用3
- 3卷积核，步长统一为1，Padding统一为1，和22最大池化核，步长为2，Padding统一为0
- 4.VGG19比VGG16的区别在于多了3个卷积层，其它完全一样
- 5.VGG16基本是AlexNet（AlexNet是8层，包括5个卷积层和3个全连接层）的加强版，深度上是其2倍，参数量大小也是两倍多。

4.3 Segnet

SegNet (**SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation**

2015CVPR) 网络是典型的编码-解码结构。SegNet编码器网络由VGG16的前13个卷积层构成，所以通常是使用VGG16的预训练权重来进行初始化。每个编码器层都有一个对应的解码器层，因此解码器层也有13层。编码器最后的输出输入到softmax分类器中，输出每个像素的类别概率。SegNet如下图所示。

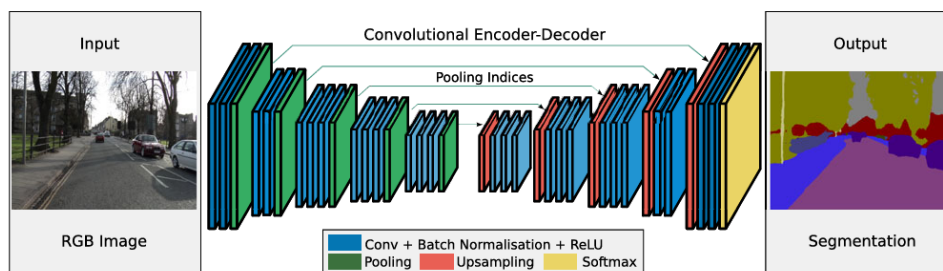


Fig18. SegNet结构

这是 SegNet 的网络结构，其中编码器含有 13 个卷积层，分别对应了 VGG16 的前 13 个卷积层，因此就可以用在 ImageNet 下训练得权重初始化网络。此外，作者丢弃了全连接层，从而保留了最深编码输出处的高分辨率特征图，同时与其他网络结构相比，大幅减少了 SegNet 编码部分的参数量（从 134 到 14.7M）。编码器中的每一层都对应了解码器中的一层，因此解码器也有 13 层。解码器的输出随后传递给一个 softmax 分类器，生成每个像素点在个类别下的独立概率。

编码器网络中的每一个 encoder 都用一组 filter 进行卷积计算，随后接一个 bn 层，再接一个 relu 激活函数。之后用一个窗口尺寸为 2，步长为 2 的 Max-Pooling，得到一个下采样结果。用 Max pooling 的原因是实现输入图像上小空间位移的平移不变性。下采样使得特征图中的每个像素点都对应了大输入图像内容。

尽管 Max Pooling 能够实现分类任务下更多的平移不变性，但这些操作也降低了特征图的空间分辨率。而这些逐渐损失的图像描述 (例如边界信息) 对于分割任务是非常重要的。因此，需要在下采样进行之前记录和保存这些边界信息。在不考虑内存的情况下，编码器中的每一层特征层都应该记录下来。但是这种方式在实际应用中是不现实的，因此本文提出了另外一种存储方式。这种方式只保存 max-pooling indices，也就是每一个窗口内的最大特征值的位置。

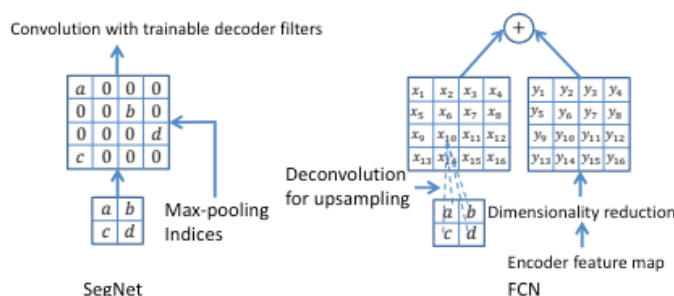


Fig. 3. An illustration of SegNet and FCN [2] decoders. a, b, c, d correspond to values in a feature map. SegNet uses the max pooling indices to upsample (without learning) the feature map(s) and convolves with a trainable decoder filter bank. FCN upsamples by learning to deconvolve the input feature map and adds the corresponding encoder feature map to produce the decoder output. This feature map is the output of the max-pooling layer (includes sub-sampling) in the corresponding encoder. Note that there are no trainable decoder filters in FCN.

在实际应用中，这个信息可以用一个 2 位的二进制数来记录一个 2×2 的窗口，因此相比较记录整个特征图而言也就更加实用。后面会证明，这种方式损失了一定的精度，但是对于实际应用还是够用了。解码器用之前记录的 max-pooling indices 上采样输入的特征图。这个过程生成的特征图是稀疏的。

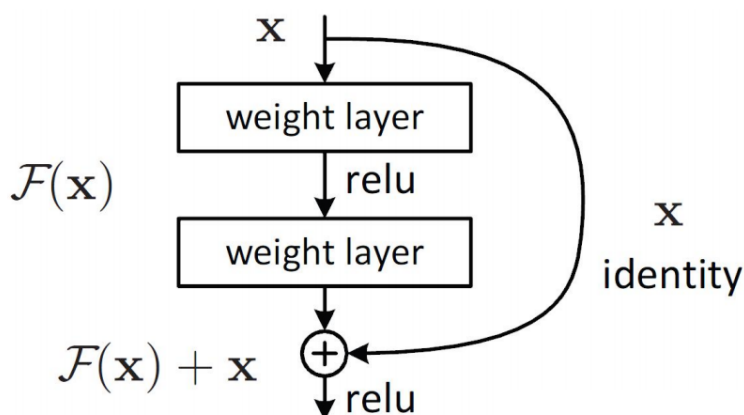
解码器的每个反卷积层后面也跟了 bn 层。与编码器第一层 (最靠近输入图像的层) 对应的解码器层有多通道 (而非完全对应的三通道)，除此之外的其他层与编码器具有相同的尺寸和通道数。解码器的输出送给一个可学习的 soft-max classifier，该分类器对每一个像素都单独分类。分类器的输出有 k 个通道，每个通道上存储的都是该类别下的概率。最终的分割结果对应的是概率最大的类别。

Segnet在参数和准确率之间找到了一个平衡点；将编解码器结构普适化，在多个场景数据集中取得了很好的效果。

4.4 Resnet

Resnet ([Deep Residual Learning for Image Recognition](#) 2016CVPR) 在深度学习历史上是里程碑式的模型，在ResNet之前，我们知道深度学习的模型一般都是在20-30层之间，但是在ResNet出现之后，把深度学习的模型层数提高到一百层以上，获得质的提升，甚至可以实现一千层以上的网络模型。

ResNet主要通过使用残差块和恒等映射解决了深层网络的退化问题。



ResNet提出了一种残差表示，将映射表示为原值和残差值之和。残差表示源自于一个朴素的想法，若每次梯度反向传播总会传递一个值恒为1的梯度，就可以有效缓解多个小梯度累乘造成的梯度消失。用数学语言表述，假设残差块输入为 x ，则输出 y 等于：

$$y = F(x, \{W_i\}) + x$$

其中 $F(x)$ 是我们学习的目标，即输入输出的残差 $y - x$ 。该设计不会增加运算量，但是能保证在梯度反向传播的时候不会发生梯度消失。这种跨层连接（shortcut connection）的残差表示是一种恒等映射（identity mapping）。

在使用了ResNet的结构后，可以发现层数不断加深导致的训练集上误差增大的现象被消除了，ResNet网络的训练误差会随着层数增加而逐渐减少，并且在测试集上的表现也会变好。原因在于，Resnet学习的是残差函数 $F(x) = H(x) - x$ ，这里如果 $F(x) = 0$ ，那么就是上面提到的恒等映射。事实上，Resnet是“shortcut connections”的，在connections是在恒等映射下的特殊情况，学到的残差为0时，它没有引入额外的参数和计算复杂度，且不会降低精度。在优化目标函数是逼近一个恒等映射，而学习的残差不为0时，那么学习找到对恒等映射的扰动会比重新学习一个映射函数要更容易。

4.5 FCN

FCN ([Fully Convilutional Networks](#) 2015CVPR) 这也是FCN的由来，即全卷积网络。

FCN的关键点主要有三，一是全卷积进行特征提取和下采样，二是双线性插值进行上采样，三是跳跃连接进行特征融合。

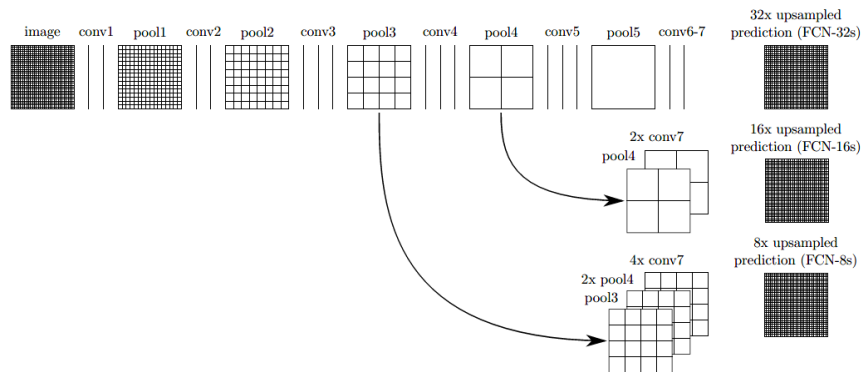


Fig15. FCN

原来的方法：卷积，采样，最后展平全连接层，对一千个像素进行softmax，最后得票最高的作为网络输出。

FCN：把全连接变成卷积，最终输出由一维变成了二维。

FCN8s中用了maxpooling3的输出，经过1*1的卷积层得到的特征层是8倍下采样大小，将FCN16s的结果再上采样2倍就可以得到一样的形状了，相加之后在进行转置卷积，8倍上采样放大即可得到原图。

所以FCN8s，FCN16s融合了下层抽象的信息。

由于使用了vgg16作为FCN-8的编码部分（backbone），这使得FCN-8具备较强的特征提取能力。

4.6 UNet

早期基于深度学习的图像分割以FCN为核心，旨在重点解决如何更好从卷积下采样中恢复丢掉的信息损失。后来逐渐形成了以UNet为核心的这样一种编解码对称的U形结构。

UNet（

U-Net: Convolutional Networks for Biomedical Image Segmentation 2015MICCAI) 结构能够在分割界具有一统之势，最根本的还是其效果好，尤其是在医学图像领域。所以，做医学影像相关的深度学习应用时，一定都用过UNet，而且最原始的UNet一般都会有一个不错的baseline表现。2015年发表UNet的MICCAI，是目前医学图像分析领域最顶级的国际会议，UNet为什么在医学上效果这么好非常值得探讨一番。

U-Net结构如下图所示：

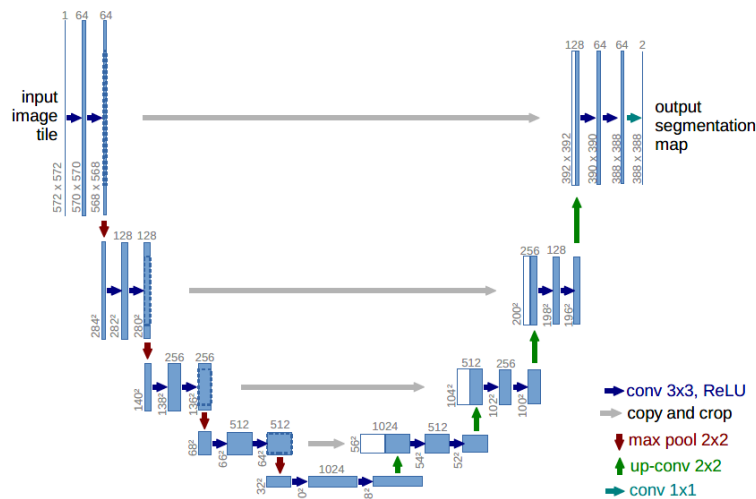


Fig16. UNet

乍一看很复杂，U形结构下貌似有很多细节问题。我们来把UNet简化一下，如下图所示：

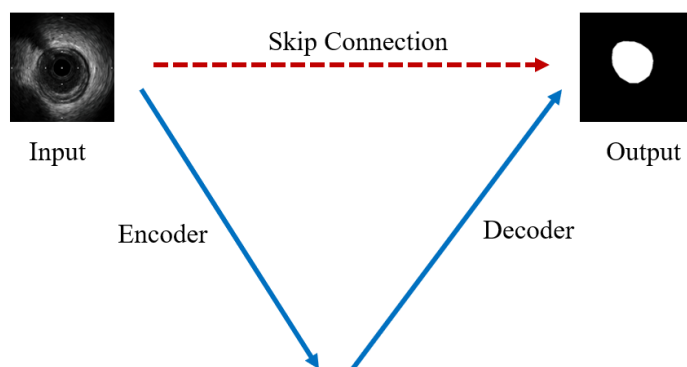


Fig17. U-Net简化

从图中可以看到，简化之后的UNet的关键点只有三条线：下采样编码，上采样解码，跳跃连接。

下采样进行信息浓缩和上采样进行像素恢复，这是其他分割网络都会有的部分，UNet自然也不会跳出这个框架，可以看到，UNet进行了4次的最大池化下采样，每一次采样后都使用了卷积进行信息提取得到特征图，然后再经过4次上采样恢复输入像素尺寸。但UNet最关键的、也是最特色的部分在于图中红色虚线的Skip Connection。每一次下采样都会有一个跳跃连接与对应的上采样进行级联，这种不同尺度的特征融合对上采样恢复像素大有帮助，具体来说就是高层（浅层）下采样倍数小，特征图具备更加细致的图特征，底层（深层）下采样倍数大，信息经过大量浓缩，空间损失大，但有助于目标区域（分类）判断，当high level和low level的特征进行融合时，分割效果往往会非常好。从某种程度上讲，这种跳跃连接也可以视为一种Deep Supervision。

4.7 Deeplab系列

Deeplab系列可以算是深度学习语义分割的另一个主要架构，其代表方法就是基于Dilation的多尺度设计。Deeplab系列主要包括：Deeplab v1 (**Semantic Image Segmentation with Deep Convolutional Nets and Fully Connected CRFs** 2014CVPR) Deeplab v2 (**DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs** 2016CVPR) Deeplab v3 (**Rethinking Atrous Convolution for Semantic Image Segmentation** 2017CVPR) Deeplab v3+ (**Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation** 2018CVPR)

Deeplab v1主要是率先使用了空洞卷积，是Deeplab系列最原始的版本。

Deeplab v2在Deeplab v1的基础上最大的改进在于提出了ASPP (Atrous Spatial Pyramid Pooling)，即带有空洞卷积的金字塔池化，该设计的主要目的就是提取图像的多尺度特征。另外Deeplab v2也将Deeplab v1的Backone网络更换为ResNet。

Deeplabv1和v2还有一个比较大的特点就是使用了CRF作为后处理技术。

这里重点说一下多尺度问题。多尺度问题就是当图像中的目标对象存在不同大小时，分割效果不佳的现象。比如同样的物体，在近处拍摄时物体显得大，远处拍摄时显得小。解决多尺度问题的目标就是不论目标对象是大还是小，网络都能将其分割地很好。Deeplabv2使用ASPP处理多尺度问题，ASPP设计结构如下图所示。

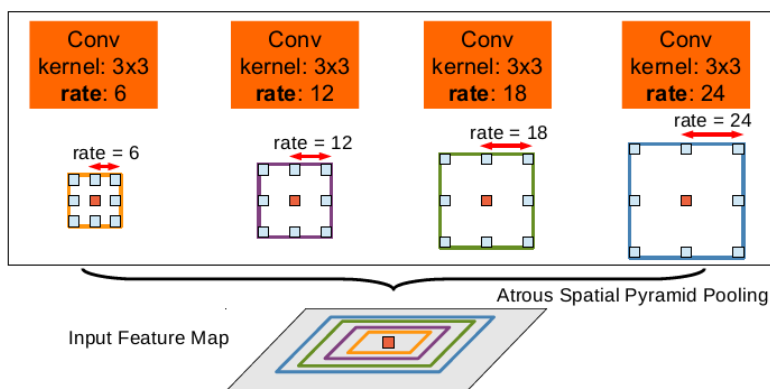


Fig19. ASPP

从Deeplab v3开始，Deeplab系列舍弃了CRF后处理模块，提出了更加通用的、适用任何网络的分割框架，对ResNet最后的Block做了复制和级联（Cascade），对ASPP模块做了升级，在其中添加了BN层。改进后的ASPP如下图所示。

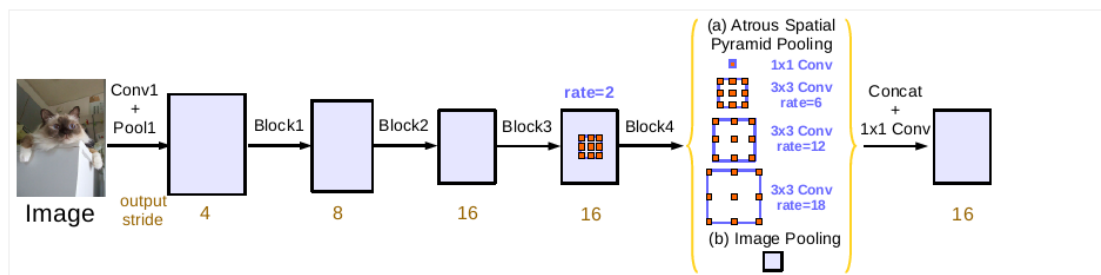


Fig20. ASPP of Deeplab v3

Deeplab v3+在Deeplab v3的基础上做了扩展和改进，其主要改进就是在编解码结构上使用了ASPP。Deeplab v3+可以视作是融合了语义分割两大流派的一项工作，即编解码+ASPP结构。另外Deeplab v3+的Backbone换成了Xception，其深度可分离卷积的设计使得分割网络更加高效。Deeplab v3+结构如下图所示。

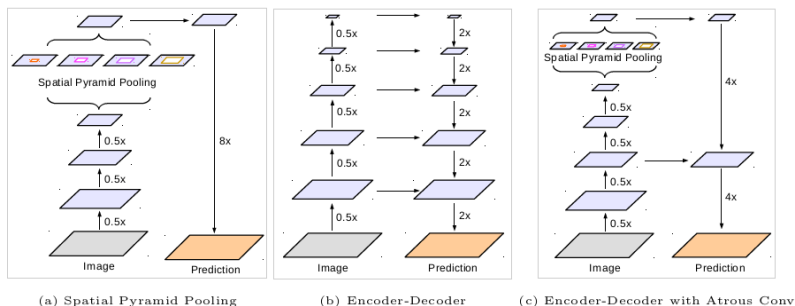


Fig21. ASPP

关于Deeplab系列各个版本的技术点构成总结如下表所示。

	Deeplab v1	Deeplab v2	Deeplab v3	Deeplab v3+
Backbone	VGG16	ResNet	ResNet+	Xception
Dilated Conv	✓	✓	✓	✓
ASPP	×	ASPP	ASPP+	ASPP+
CRF	✓	✓	×	×
Encoder-Decoder	×	×	×	✓

Fig22. Summary of Deeplab series.

4.8 PSPNet

PSPNet (**Pyramid Scene Parsing Network** 2017CVPR) 是针对多尺度问题提出的另一种代表性分割网络。PSPNet认为此前的分割网络没有引入足够的上下文信息及不同感受野下的全局信息而存在分割出现错误的情况，因而引入Global-Scence-Level的信息解决该问题，其Backbone网络也是ResNet。简单来说，PSPNet就是将Deeplab的ASPP模块之前的特征图Pooling了四种尺度，然后将原始特征图和四种Pooling之后的特征图进行合并到一起，再经过一系列卷积之后进行预测的过程。PSPNet结构如下图所示。

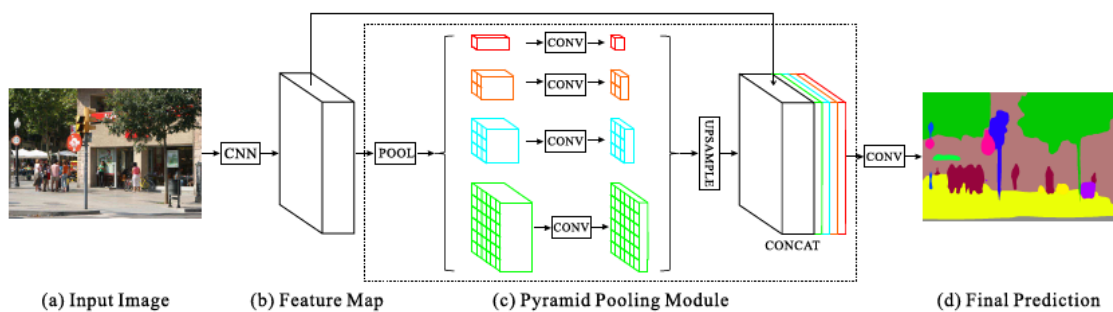


Fig23. PSPNet

一个简易的带有深监督的PSPNet PPM模块写法如下：

```
# Pyramid Pooling Module
class PPMDeepsup(nn.Module):
    def __init__(self, num_class=150, fc_dim=4096,
                  use_softmax=False, pool_scales=(1, 2, 3, 6)):
        super(PPMDeepsup, self).__init__()
        self.use_softmax = use_softmax
        # PPM
        self.ppm = []
        for scale in pool_scales:
            self.ppm.append(nn.Sequential(
                nn.AdaptiveAvgPool2d(scale),
                nn.Conv2d(fc_dim, 512, kernel_size=1, bias=False),
                BatchNorm2d(512),
                nn.ReLU(inplace=True)
            ))
        self.ppm = nn.ModuleList(self.ppm)
        # Deep Supervision
        self.cbr_deepsup = conv3x3_bn_relu(fc_dim
// 2, fc_dim // 4, 1)
        self.conv_last = nn.Sequential(
            nn.Conv2d(fc_dim+len(pool_scales)*512, 512,
                      kernel_size=3, padding=1, bias=False),
            BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.1),
            nn.Conv2d(512, num_class, kernel_size=1)
        )
        self.conv_last_deepsup = nn.Conv2d(fc_dim // 4, num_class, 1, 1, 0)
        self.dropout_deepsup = nn.Dropout2d(0.1)
```

4.9 UNet++

自从2015年UNet网络提出后，这么多年大家没少在这个U形结构上折腾。大部分做语义分割的朋友都没少在UNet结构上做各种魔改，如果把UNet++算作是UNet的一种魔改的话，那它一定是最成功的魔改者。

UNet++ (

UNet++: A Nested U-Net Architecture for Medical Image Segmentation 2018MICCAI) 是一种嵌套的U-Net结构，即内置了不同深度的UNet网络，并且利用了全尺度的跳跃连接 (skip connection) 和深度监督 (deep supervisions)。另外UNet++还设计一种剪枝方案，加快了UNet++的推理速度。UNet++的结构示意图如下所示。

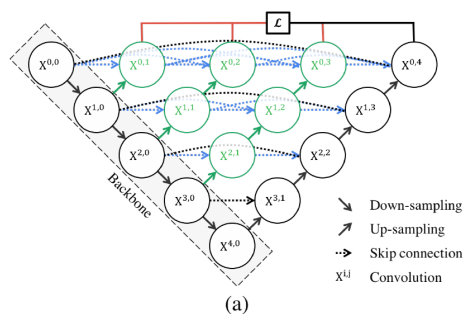


Fig24. UNet++

单纯从结构设计角度来看，UNet++效果好要归功于其嵌套结构和重新设计的跳跃连接，旨在解决UNet的两个关键挑战:优化整体结构的未知深度和跳跃连接的不必要的限制性设计。

从 UNet++ 开始，网络的设计就开始注意原始图像的位置信息，还有就是整个网络的信息融合的能力。所以，UNet++ 通过一系列尝试，最终设计成稠密连接，同时有长连接和短连接，并且将 UNet 中间空心的位置填满；可以抓取不同层次的特征 / 不同大小的感受野。UNet++ 横向看可以看做一个稠密结构的 DenseNet。

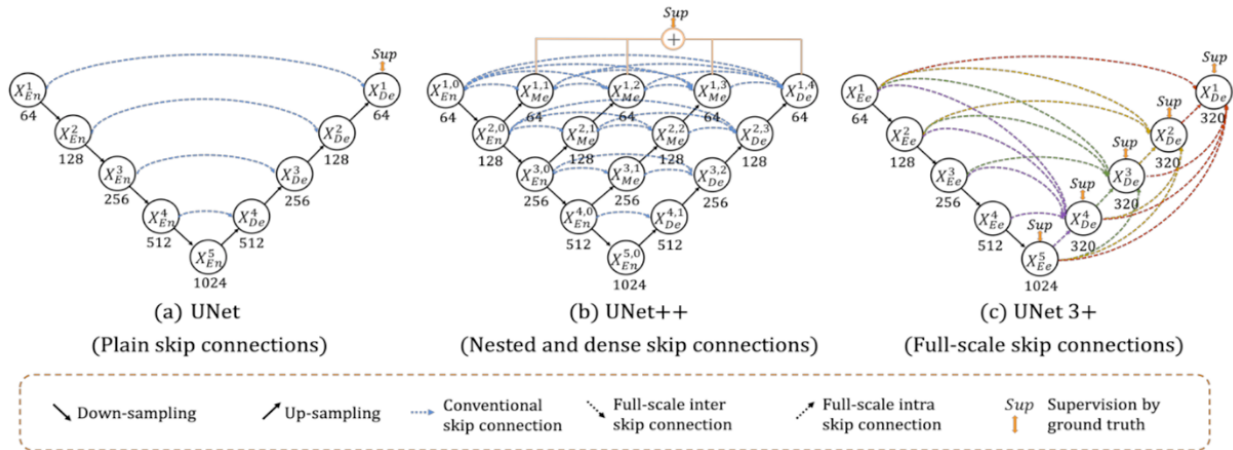
完整实现过程可参考[GitHub](#)开源代码。

4.10 Unet3+

Unet++的问题：Unet++通过设计具有嵌套和密集跳过连接的架构来修改UNET的，然而，它没有从全尺度探索足够的信息。

Unet3+ (

UNet 3+: A Full-Scale Connected UNet for Medical Image Segmentation 2020ICASSP) 在Unet++的基础上继续改进，通过引入全尺度的跳过连接，在全尺度特征映射中融合了低层细节和高层语义，充分利用了多尺度特征的同时具有更少的参数；通过深度监督让网络从全尺度特征中学习分割表示，提出了更优的混合损失函数以增强器官的边界；提出分类指导模块，通过与图像分类分支联合训练的方式，减少了网络在非器官图像的过度分割。



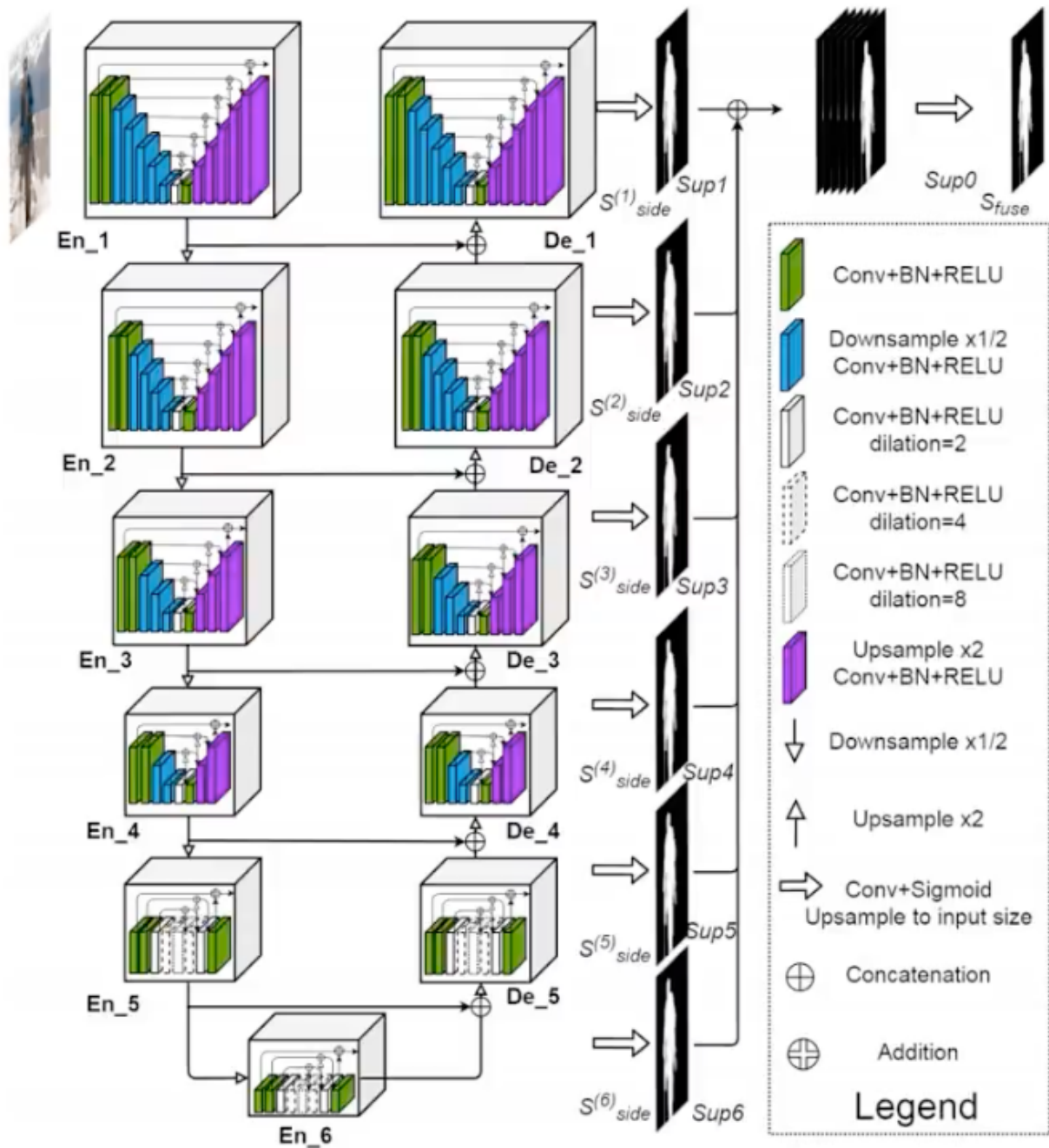
UNet++ 只是针对于同一尺度的稠密连接，而 UNet3 + 则是跨尺度的稠密连接；UNet3 + 横纵信息互相交融，像极了国内高铁的“八横八纵”的高铁网，可以获得更大范围的信息融合与流通。又一次感觉很多算法上的设计思想都有异曲同工之妙，亦或者说是源于生活。

UNet3 + 效果在医疗图像上分割有着不俗的效果，还有一部分是来源于其精心设计的损失函数 MS-SSIM，其作为 UNet3 + 组合损失函数的一部分，也会在提点上有着一定的作用（算法除了模型设计，当然还逃脱不了损失的设计）。

4.11 U2net

U2net (**U2-Net: Going Deeper with Nested U-Structure for Salient Object Detection** 2020CVPR) 是在Unet的基础上改进，算另一种改进思路。

U2net利用来自残差U型模块（RSU）的不同尺度不同感受野的混合，能够捕捉来自更多的不同尺度的上下文信息（全局信息）；采用RSU中的池化操作，U2Net在不增加计算复杂度的基础上，可以提升整个模型架构的深度。



嵌套 UNet 的方式，不可谓不是一种天秀的方法；同时通过膨胀卷积的使用，在不增加计算量的情况下，获得了很大的感受野，整个网络看起来变得很深，但是在参数量上的控制，已经达到了一种炉火纯青的地步。不过，这篇论文的实验的数据集不是在针对医疗图像，也没有去做实验去比较。但是，U2Net 为了获得更大的感受野，在位置信息上丢失的还是比较多的，在对于位置信息的表现力，可能有所欠缺。

4.12 Swin-Transformer

Swin-Transformer (Swin Transformer: Hierarchical Vision Transformer using Shifted Windows 2021ICCV) 是2021年微软研究院发表在ICCV上的一篇文章，并且已经获得ICCV 2021 best paper的荣誉称

号。Swin Transformer网络是Transformer模型在视觉领域的又一次碰撞。

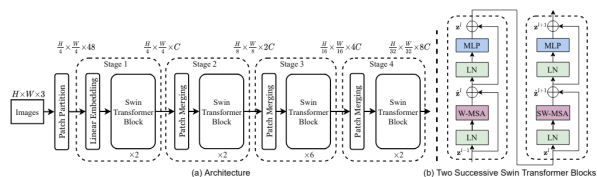
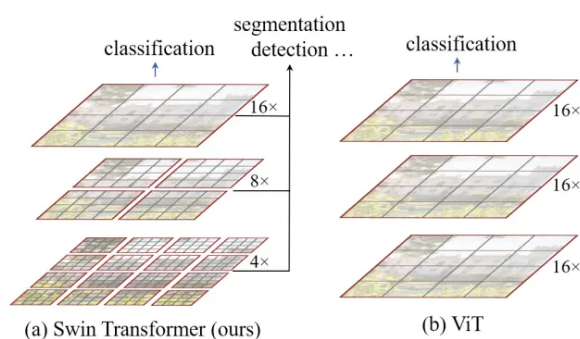


Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

通过与CNN相似的分层结构来处理图片，使得模型能够灵活处理不同尺度的图片；采用了window self-attention，降低了计算复杂度。

模型主要分为三个部分：图像处理——将RGB像素分辨率的图像转化为patches分辨率图像并根据具体模型大小改变输入通道数；Swin Transformer stage——实现window self-attention及shifted window self-attention，通过Patch Merging实现分层结构，降低计算复杂度并能够处理不同尺度的图片；不同视觉任务输出。

Swin Transformer的提出解决了之前Transformer应用在CV领域的缺点，具有替换掉CNN成为新的视觉任务Backbone的潜力，通过Patch Merging结构和W-MSA/SW-MSA在不同的视觉任务上取得了SOTA的效果。

4.13 Swin-Unet

在医学图像分割领域，U型网络结构是默认选项，大多是使用CNN构建Unet，当然也有TransUNet这种融合CNN和Transformer的Unet，作者更进一步，看到Swin Transformer在众多任务上取得的良好效果后，提出了Swin-Unet ([Swin-Unet: Unet-like Pure Transformer for Medical Image Segmentation ECCV 2022](#))，只用Swin Transformer来构建U型网络做2D医学图像分割。

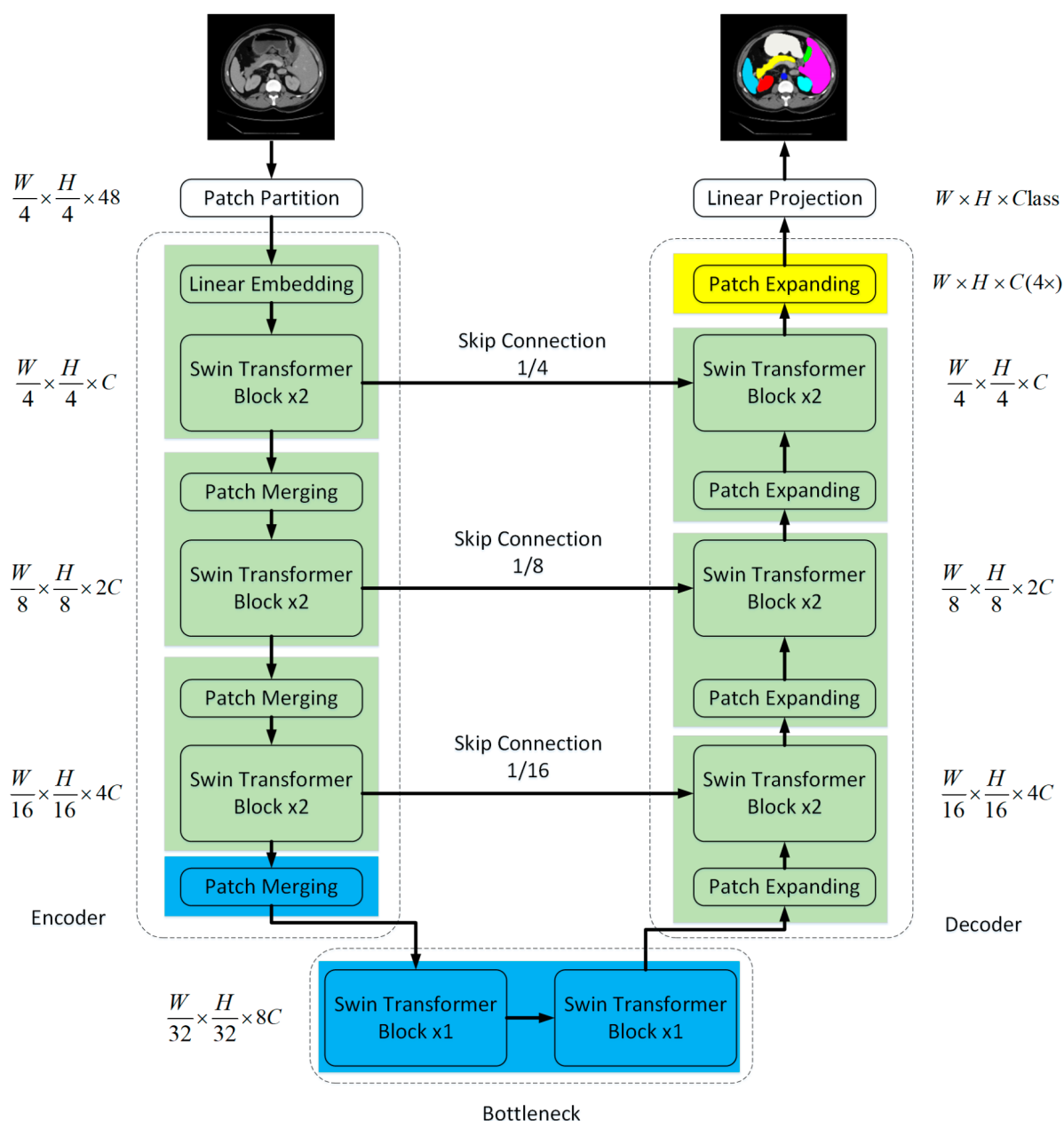


Fig. 1. The architecture of Swin-Unet, which is composed of encoder, bottleneck, decoder and skip connections. Encoder, bottleneck and decoder are all constructed based on swin transformer block.

Swin-UNet由Encoder、Bottleneck、Decoder和跳跃连接组成。先看编码器部分，输入图像先进行patch partition，每个patch大小为4x4，输入维度为 $H/4 \times W/4 \times 48$ ，经过linear embedding和两个Swin Transformer block（一个Swin Transformer block由一个W-MSA和一个SW-MSA组成。）后特征图尺寸为 $H/4 \times W/4 \times C$ ，然后通过patch merging进行下采样，再经过两个Swin Transformer block后特征图尺寸变

为 $H/8 \times W/8 \times 2C$ ，最后再进行一次同样的下采样操作即可完成编码器的操作。可以看到，Swin-UNet编码器每次按照2倍来缩小patch的数量，然后按照3倍来扩大特征维度的数量。

Bottleneck则是用了两个连续的Swin Transformer block，这里为防止网络太深不能收敛，所以只用了两个block，在Bottleneck中，特征尺寸保持 $H/32 \times W/32 \times 8C$ 不变。

然后是解码器部分。Swin-UNet解码器主要由patch expanding来实现上采样，作为一个完全对称的网络结构，解码器也是每次扩大2倍进行上采样，核心模块由Swin Transformer block和patch expanding组成。

Patch expanding layer：以第一个patch扩展层为例，在上采样之前，对输入特征($W/32 \times H/32 \times 8C$)加一个线性层，将特征维数增加到原维数的2倍($W/32 \times H/32 \times 16C$)。然后，利用rearrange运算将输入特征的分辨率扩展到输入分辨率的2倍，并将特征维数缩减到输入维数的 $1/4$ ($w/32 \times h/32 \times 16C \rightarrow w/16 \times h/16 \times 4C$)。

最后是跳跃连接。跳跃连接可以算是UNet的特色，Swin-UNet也自然不例外。

贡献：

- (1)基于Swin Transformer模块构建了一个带有跳过连接的对称编码器-解码器架构。该编码器实现了从局部到全局的自注意;在解码器中，将全局特征上采样到输入分辨率，进行相应的像素级分割预测。
- (2)在不使用卷积和插值运算的情况下，设计了一种patch扩展层，实现了上采样和特征维数的增加。
- (3)实验发现跳跃连接对Transformer也是有效的，因此最终构造了一个基于Transformer的带跳跃连接的U型编解码器体系结构，命名为Swin-Unet。

4.14 UTNet

在语义分割上，FCN 这类卷积的编码器 - 解码器架构衍生出的模型在过去几年取得了实质性进展，但这类模型存在两个局限。**第一**，卷积仅能从邻域像素收集信息，缺乏提取明确全局依赖性特征的能力；**第二**，卷积核的大小和形状往往是固定的，它们不能灵活适应输入的图像或其他内容。

为了解决上面的问题，U-Net 混合 Transformer 网络：UTNet ([UTNet: A Hybrid Transformer Architecture for Medical Image Segmentation MICCAI 2021](#))，它整合了卷积和自注意力策略用于医学图像分割任务。应用卷积层来提取局部强度特征，以避免对 Transformer 进行大规模的预训练，同时使用自注意力来捕获全局特征。为了提高分割质量，还提出了一种 efficient self-attention，在时间和空间上将整体复杂度从 $O(n^2)$ 显著降低到接近 $O(n)$ 。此外，在 self-attention 模块中使用相对位置编码来学习医学图像中的内容 - 位置关系。

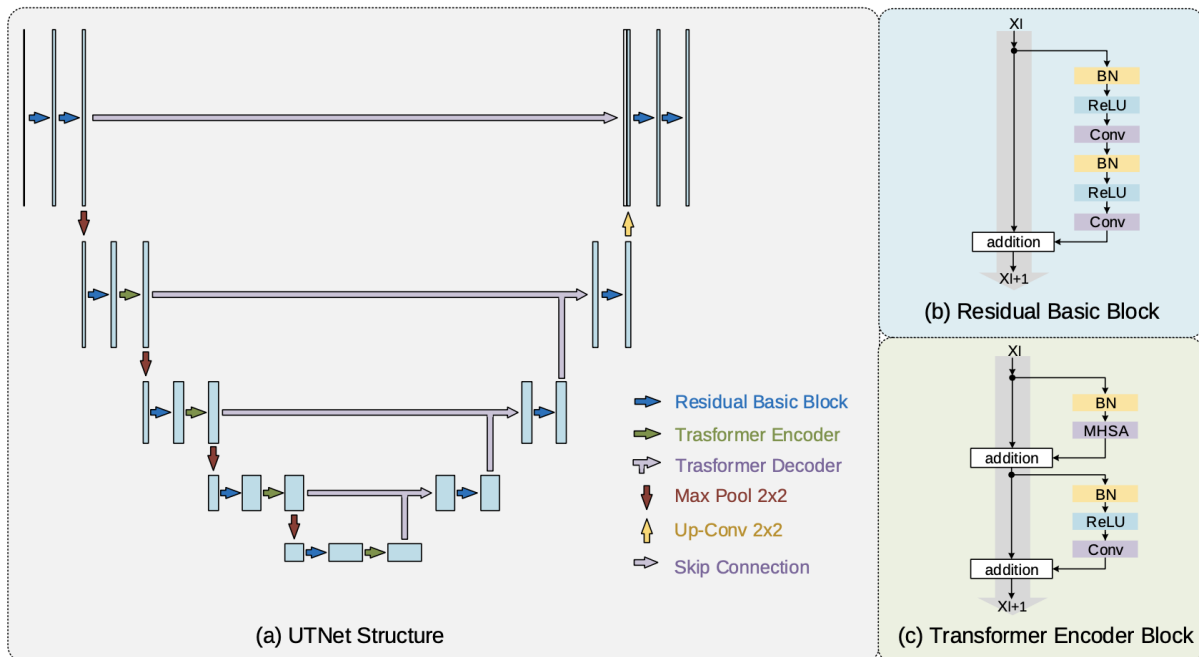
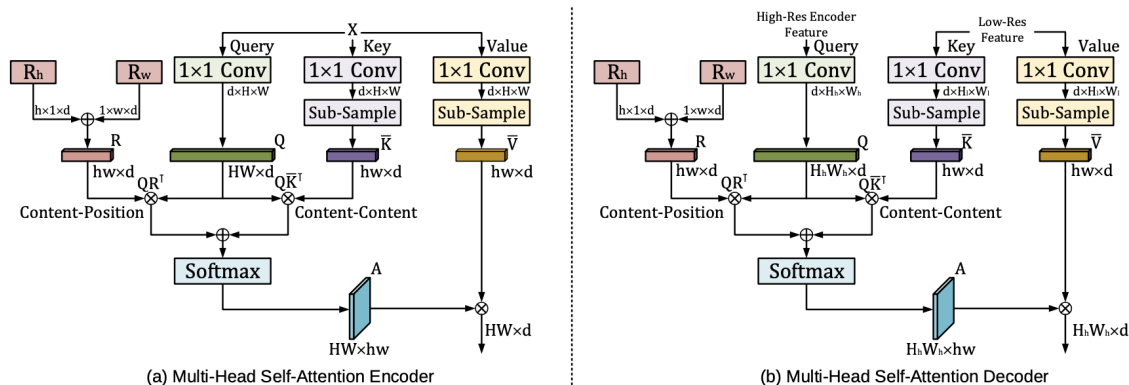


Fig. 1. (a) The hybrid architecture of the proposed UTNet. The proposed efficient self-attention mechanism and relative positional encoding allow us to apply Transformer to aggregate global context information from multiple scales in both encoder and decoder. (b) Pre-activation residual basic block. (c) The structure of Transformer encoder block.



如上图所示 UTNet 结构图，整体上还是保持 U 型。(b) 是一个经典的残差块，传统的 U-Net 改进方法也是这么做的，这样也可以提高分割任务的准确率，避免网络深度带来的梯度爆炸和梯度消失等问题，这些都是老生常谈了，我们不重点关注。(c) 是一个标准的 Transformer Decoder 设计。可以发现，遵循了 U-Net 的标准设计，但将每个构建块的最后一个卷积（最高的除外）替换为 Transformer 模块。此外，低三层的跨层连接也被替换为了 Transformer Decoder。

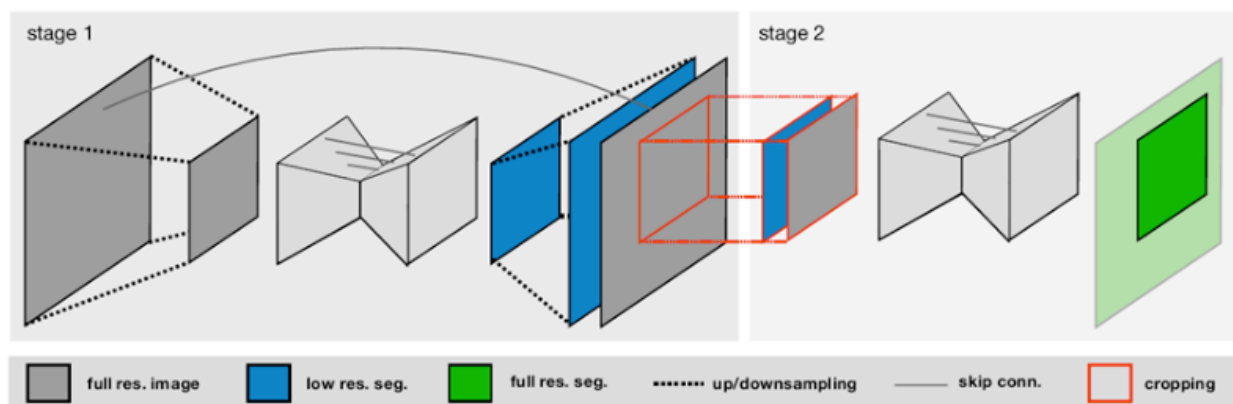
这种混合架构可以利用卷积图像的归纳偏差来避免大规模预训练，以及 Transformer 捕获全局特征关系的能力。由于错误分割的区域通常位于感兴趣区域的边界，高分辨率的上下文信息可以在分割中发挥至关重要的作用。因此，重点放在了自我注意模块上，这使得有效处理大尺寸特征图成为可能。没有将自注意力模块简单地集成到来自 CNN 主干的特征图之上，而是将 Transformer 模块应用于编码器和解码器的每个级别，以从多个尺度收集长期依赖关系。请注意，没有在原始分辨率上应用 Transformer，因为在网络的非常浅层中添加

Transformer 模块对实验没有帮助，但会引入额外的计算。一个可能的原因是网络的浅层更多地关注详细的纹理，其中收集全局上下文特征效果肯定不理想。

贡献：UTNet将自注意力集成到了卷积神经网络中；提出了一种高效的自注意力算法；混合层的设计允许将Transformer初始化为卷积网络，无需预训练。

4.15 nnUnet

相较于之前的各种UNet改进，nnUNet ([nnU-Net: Self-adapting Framework for U-Net-Based Medical Image Segmentation](#) 2021NIT) 更注重图像的预处理工作，能够自动判断影像模态并进行与之对应的归一化操作，并且根据三次样条插值对不同的图像体素间距进行重采样。



网络结构方面，nnUNet基于原始的UNet提出了3个网络，分别是2D UNet、3D UNet和2个级联的3D UNet，第一个用于生成粗分割结果，第二个则用于细化粗分割结果，如图3所示。3个UNet模型能够彼此独立的进行配置、设计和训练，相较于原始的UNet，nnUNet对其结构作了微调：使用填充卷积来实现相同的输出和输入形状，使用Leaky ReLU代替ReLU，使用实例归一化来代替批量归一化。nnUNet能够自动设置超参数，比如训练批次大小、图像分块大小、下采样次数等。所有的UNet架构均通过五折交叉验证，使用交叉熵损失和Dice损失作为训练时的损失函数，优化器使用Adam，并设置学习率动态调整策略，同时训练时也使用在线的数据增强策略。

nnU-Net使用简单的U-Net架构，但通过良好的配置超越了更复杂的方法，相当于tricks的集大成者。

4.16 TransUnet

U-Net以及经过各种五花八门魔改的U-Net目前称霸着医学图像分割领域。但是U-Net不管怎么魔改，只要用的算子依旧是卷积，就会存在卷积的局限性。U-Net在明确建立远程依赖上存在局限性。而Transformer虽然近几年成了一个新的网红算子，但是在细节上的把控不足，导致了局部特征学习的局限。本文将U-Net和Transformer结合，提出TransUNet ([TransUNet: Transformers Make Strong Encoders for Medical Image Segmentation](#) CVPR 2021)。

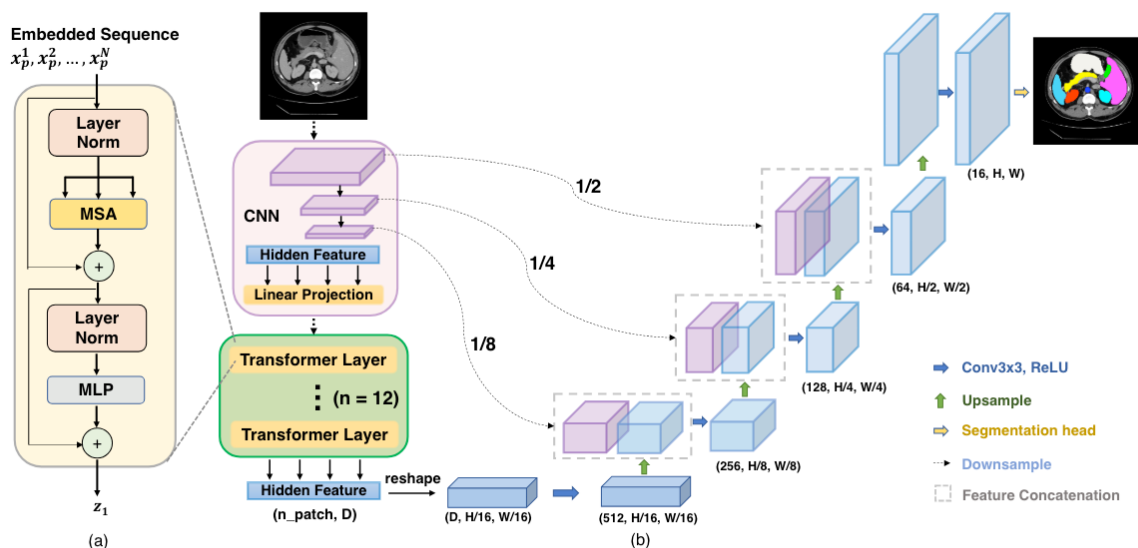


Fig. 1: Overview of the framework. (a) schematic of the Transformer layer; (b) architecture of the proposed TransUNet.

首先看到ViT的部分。和ViT一样，把图片切成一个个patch然后经过线性映射变成D维的embedding。此外，我们在patch embedding上加上position embedding记录位置信息。然而，单纯这样的架构依然会造成细节信息损失。所以我们在transformer之前加上一些CNN用来提取局部信息，并且加上了cascaded upsampler更精确地消化局部信息。杂交encoder首先使用CNN提取局部信息，然后把得到的feature进行1×1的patch切分，放入transformer里。每一个Upsampling Block里都有2个upsampling操作，一个3×3卷积，和一层ReLU。同时，transformer之前的CNN每一层的feature也都和upsampling中每一层的feature连接。

Transunet的设计思想也相对简单，纯粹是把U-net里的encoder部分套上了vision transformer。目前难点在于如何降低模型参数量的同时，保持高分割精度。

4.17 TransDeeplab

Deeplab V3+通过添加一个简单有效的解码器提升了分割性能，但并没有解决CNN的根本性缺点，即不可避免的在学习长程依赖性和空间相关性方面的限制。本文提出了TransDeeplab ([TransDeepLab: Convolution-Free Transformer-based DeepLab v3+ for Medical Image Segmentation Springer 2022](#)) 旨在使用swin-Transformer来模拟经典的Deeplab V3+。

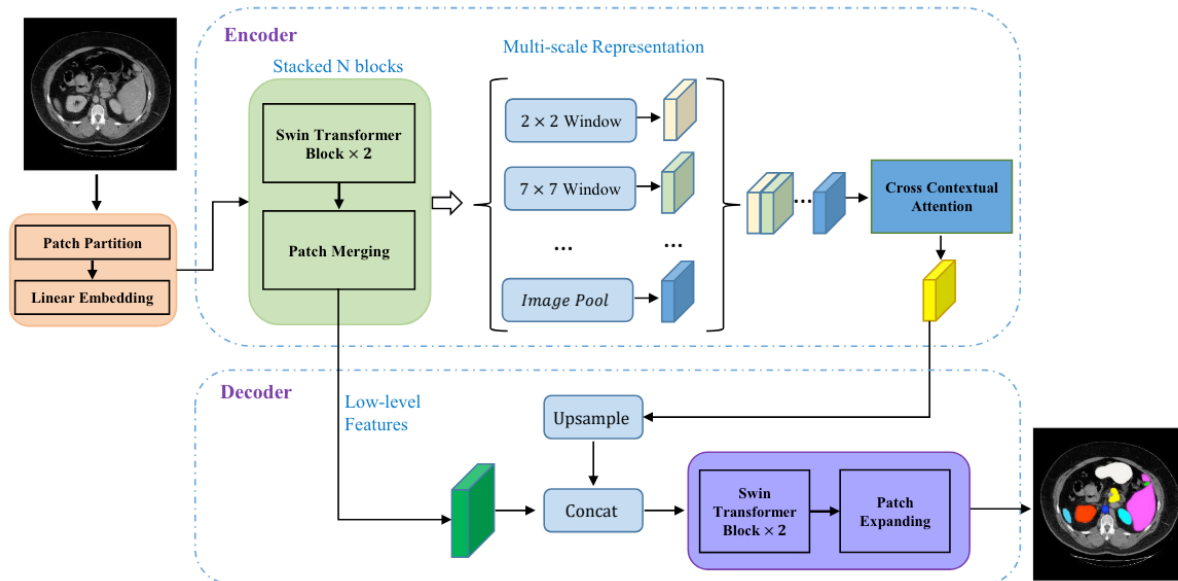


Fig. 1. The architecture of TransDeepLab, which extends the encoder-decoder structure of DeepLabv3+. Encoder and decoder are all constructed based on Swin-Transformer blocks.

具体来说，编码器模块将输入的医学图像分割成大小为 4×4 的非重叠补丁，每个补丁的特征维度为 $4 \times 4 \times 3 = 48$ （表示为C），并应用Swin-Transformer块来编码局部语义和远程上下文表示。为了建模Atrous空间金字塔池化（ASPP），设计了一组具有不同窗口大小的Swin-Transformer块的金字塔。Swin金字塔的主要思想是通过利用不同的窗口大小来捕获多尺度信息。然后，获得的多尺度上下文表示通过交叉上下文注意机制融合到解码器模块中。注意块在标记上应用两级注意（例如，通道和空间注意）来制定多尺度交互。最后，在解码路径中，提取的多尺度特征首先进行双线性上采样，然后与编码器的低级特征连接，以精化特征表示。

受Swin-Transformer块低计算负担的启发，以及其在建模长程上下文依赖性方面的优势（不同于常规CNN），我们使用堆叠的Swin-Transformer模块来建模我们的编码器模型。我们的TransDeepLab编码器首先将分辨率为 $H/4 \times W/4$ 的C维标记化输入馈送到两个连续的Swin-Transformer块中，以生成分层表示，同时保持分辨率不变。然后，它应用一系列堆叠的Swin-Transformer块逐渐减少特征图的空间维度（类似于CNN编码器），并增加特征维度。然后，将结果的中间级表示馈送到Swin空间金字塔池化（SSPP）块以捕获多尺度表示。

由于堆叠的Swin-Transformer块后跟的补丁合并层（类似于CNN编码器中的连续下采样操作），编码器模块提取的深度特征的空间分辨率显著降低。

因此，为了补偿空间表示并产生多尺度表示，DeepLab模型利用ASPP模块，该模块用空洞卷积替换了池化操作。具体而言，DeepLab旨在通过应用具有多个空洞率的并行卷积操作来形成金字塔表示。为了以纯Transformer方式建模这样的操作，我们创建了一个具有不同窗口大小的Swin空间金字塔池化（SSPP）块，以捕获多尺度表示。在我们的设计中，较小的窗口大小旨在捕获局部信息，而较大的窗口则用于提取全局信息。然后，将得到的多尺度表示馈送到一个跨上下文注意力模块，以非线性技术融合和捕获通用表示。

在解码器中，首先将与注意力模块对应的获取的特征，通过SwinTransformer块进行处理，执行补丁扩展操作以将其上采样4倍，然后与低级

特征进行拼接。将浅层特征和深层特征拼接在一起的方案有助于通过下采样层减少空间细节的丢失。最后，一系列级联的Swin-Transformer块与路径扩展操作被应用以达到 $H \times W$ 的完整分辨率。

贡献：通过将分层swin-Transformer的优势融入deeplab架构；交叉上下文注意力能自适应融合多尺度表示。

4.18 ConvNet（待施工）

4.19 MedNeXt（待施工）

以上仅对几个主要的语义分割网络模型进行介绍，从当年的FCN到如今的各种模型层出不穷，想要对所有的SOTA模型全部进行介绍已经不太可能。其他诸如ENet、DeconvNet、RefineNet、HRNet、PixelNet、BiSeNet、UpperNet等网络模型，均各有千秋。本小节旨在让大家熟悉语义分割的主要模型结构和设计。深度学习和计算机视觉发展日新月异，一个新的SOTA模型出来，肯定很快就会被更新的结构设计所代替，重点是我们要了解语义分割的发展脉络，对主流的前沿研究能够保持一定的关注。

5. 语义分割训练Tips

PyTorch是一款极为便利的深度学习框架。在日常实验过程中，我们要多积累和总结，假以时日，人人都能总结出一套自己的高效模型搭建和训练套路。这一节我们给出一些惯用的PyTorch代码搭建方式，以及语义分割训练过程中的可视化方法，方便大家在训练过程中能够直观的看到训练效果。

5.1 PyTorch代码搭建方式

无论是分类、检测还是分割抑或是其他非视觉的深度学习任务，其代码套路相对来说较为固定，不会跳出基本的代码框架。一个深度学习的实现代码框架无非就是以下五个主要构成部分：

数据：Data；模型：Model；判断：Criterion；优化：Optimizer；日志：Logger

所以一个基本的顺序实现范式如下：

```
# datadataset = VOC()|COCO()|ADE20K()
data_loader = data.DataLoader(dataSet)
# modelmodel = ...
model_parallel = torch.nn.DataParallel(model)
# Criterionloss = criterion(...)
# Optimizeroptimer = optim.SGD(...)
# Logger and Visulizationvisdom = ...
tensorboard = ...
textlog = ...
# Model Parametersdata_size, batch_size, epoch_size, iterations = ..., ...
```

不论是哪种深度学习任务，一般都免不了以上五项基本模块。所以一个简单的、相对完整的PyTorch模型项目代码应该是如下结构的：

```
|-- semantic segmentation example
    |-- dataset.py
```

```

|-- models
|   |-- unet.py
|   |-- deeplabv3.py
|   |-- pspnet.py
|   |-- ...
|-- _config.yml
|-- main.py
|-- utils
|   |-- visual.py
|   |-- loss.py
|   |-- ...
|-- README.md
...

```

上面的示例代码结构中，我们把训练和验证相关代码都放到 `main.py` 文件中，但在实际实验中，这块的灵活性极大。一般来说，模型训练策略有三种，一种是边训练边验证最后再测试、另一种则是在训练中验证，将验证过程糅合到训练过程中，还有一种最简单，就是训练完了再单独验证和测试。所以，我们这里也可以单独定义对应的函数，训练 `train()`、验证 `val()` 以及测试 `test()` 除此之外，还有一些辅助功能需要设计，包括打印训练信息 `print()`、绘制损失函数 `plot()`、保存最优模型 `save()`，调整训练参数 `update()`。

所以训练代码控制流程可以归纳为TVT+PPSU的模式。

5.2 可视化方法

PyTorch原生的可视化支持模块是Visdom，当然鉴于TensorFlow的应用广泛性，PyTorch同时也支持TensorBoard的可视化方法。语义分割需要能够直观的看到训练效果，所以在训练过程中辅以一定的可视化方法是十分必要的。

5.2.1 Visdom

visdom是一款用于创建、组织和共享实时大量训练数据可视化的灵活工具。深度学习模型训练通常放在远程的服务器上，服务器上训练的一个问题就在于不能方便地对训练进行可视化，相较于TensorFlow的可视化工具TensorBoard，visdom则是对应于PyTorch的可视化工具。直接通过 `pip install visdom` 即可完成安装，之后在终端输入如下命令即可启动visdom服务：

```
python -m visdom.server
```

启动服务后输入本地或者远程地址，端口号8097，即可打开visdom主页。具体到深度学习训练时，我们可以在torch训练代码下插入visdom的可视化模块：

```

if args.steps_plot > 0 and step % args.steps_plot == 0:
    image = inputs[0].cpu().data
    vis.image(image, f'input (epoch: {epoch}, step: {step})')
    vis.image(outputs[0].cpu().max(0)[1].data, f'output (epoch: {epoch}, step: {step})')
    vis.image(targets[0].cpu().data, f'target (epoch: {epoch}, step: {step})')

```

```
p}))
vis.image(loss, f'loss (epoch: {epoch}, step: {step})')
```

visdom效果展示如下：

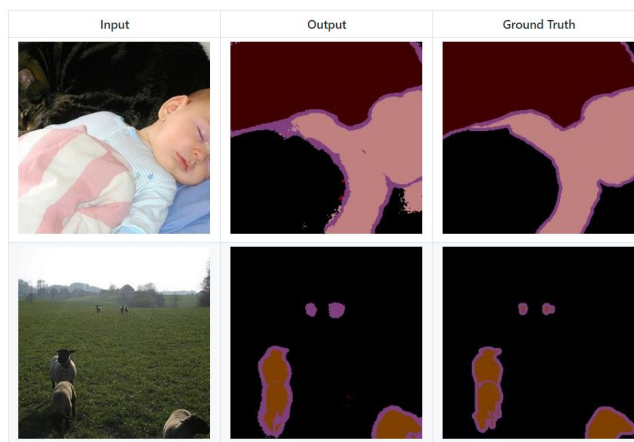


Fig25. visdom example

5.2.2 TensorBoard

很多TensorFlow用户更习惯于使用TensorBoard来进行训练的可视化展示。为了能让PyTorch用户也能用上TensorBoard，有开发者提供了PyTorch版本的TensorBoard，也就是tensorboardX。熟悉TensorBoard的用户可以无缝对接到tensorboardX，安装方式为：

```
pip install tensorboardX
```

除了要安装PyTorch之外，还需要安装TensorFlow。跟TensorBoard一样，tensorboardX也支持scalar, image, figure, histogram, audio, text, graph, onnx_graph, embedding, pr_curve, video等不同类型的对象的可视化展示方式。tensorboardX和TensorBoard的启动方式一样，直接在终端下运行：

```
tensorboard --logdir runs
```

一个完整tensorboardX使用demo如下：

```
import torch
import torchvision.utils as vutils
import numpy as np
import torchvision.models as models
from torchvision import datasets
from tensorboardX import SummaryWriter
resnet18 = models.resnet18(False)
writer = SummaryWriter()
sample_rate = 44100
freqs = [262, 294, 330, 349, 392, 440, 440, 440, 440, 440, 440]
```

```

for n_iter in range(100):
    dummy_s1 = torch.rand(1)
    dummy_s2 = torch.rand(1)
    # data grouping by `slash`
    writer.add_scalar('data/scalar1', dummy_s1
[0], n_iter)
    writer.add_scalar('data/scalar2', dummy_s2[0], n_iter)
    writer.add_scalars('data/scalar_group', {'xsinx': n_iter * np.sin(n_ite
r),
                                             'xcosx': n_iter * np.cos(n_ite
r),
                                             'arctanx': np.arctan(n_iter)}, n
_iter)
    dummy_img = torch.rand(32, 3, 64, 64) # output from network    if n_iter
% 10 == 0:
        x = vutils.make_grid(dummy_img, normalize=True, scale_each=True)
        writer.add_image('Image', x, n_iter)
        dummy_audio = torch.zeros(sample_rate * 2)
        for i in range(x.size(0)):
            # amplitude of sound should in [-1, 1]
            dummy_audio[i]
= np.cos(freqs[n_iter // 10] * np.pi * float(i) / float(sample_rate))
        writer.add_audio('myAudio', dummy_audio, n_iter, sample_rate=sample_r
ate)
        writer.add_text('Text', 'text logged at step:' + str(n_iter), n_iter)
        for name, param in resnet18.named_parameters():
            writer.add_histogram(name, param.clone().cpu().data.numpy(), n_it
er)
            # needs tensorboard 0.4RC or later
            writer.add_pr_curve('xoxo',
np.random.randint(2, size=100), np.random.rand(100), n_iter)
        dataset = datasets.MNIST('mnist', train=False, download=True)
        images = dataset.test_data[:100].float()
        label = dataset.test_labels[:100]
        features = images.view(100, 784)
        writer.add_embedding(features, metadata=label, label_img=images.unsqueeze(1))
        # export scalar data to JSON for external processing
        writer.export_scalars_to_
json("./all_scalars.json")
        writer.close()

```

tensorboardX的展示界面如图所示。

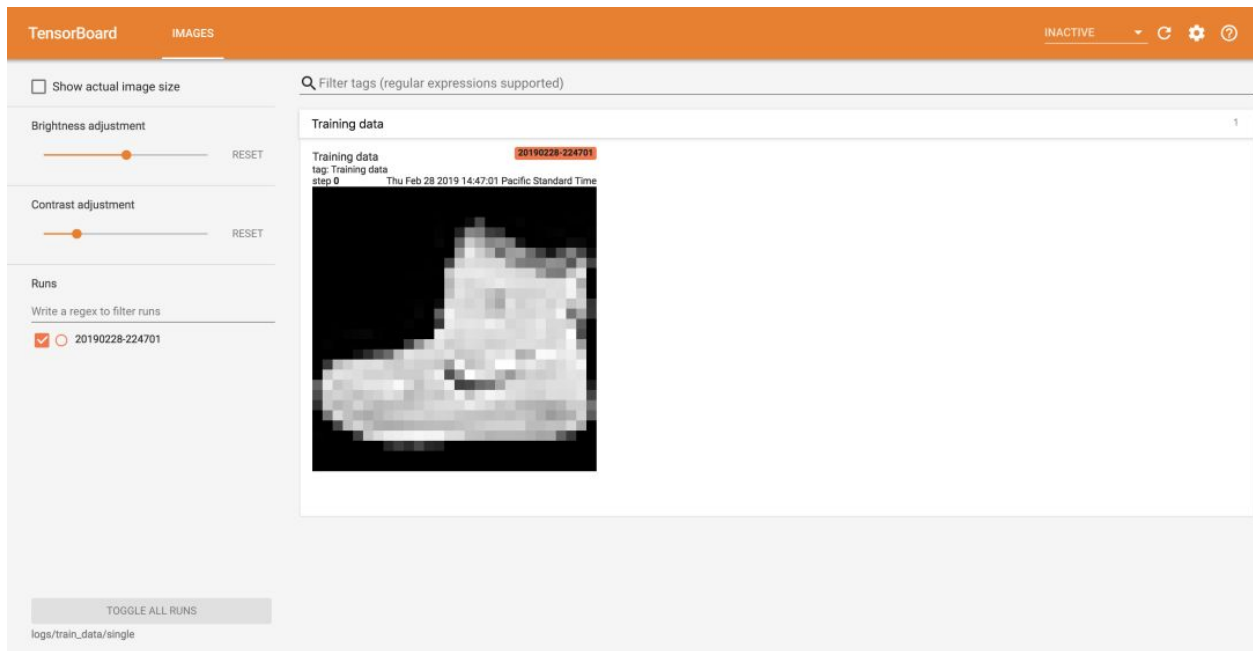


Fig26. tensorboardX example